Tugas Besar 1
IF3070 Dasar Inteligensi Artifisial
Implementasi Algoritma Local Search untuk Penataan Kendaraan pada Geladak Kapal



Kelompok 28

Anggota:

Muthia Ariesta Anggraeni	18224045
Benedicta Sherin Chyntia P.	18224065
Aisyah Az Zahra	18224121

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
SISTEM DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI **2**
DESKRIPSI PERSOALAN **3**
PEMBAHASAN **5**
A. Fungsi Initial State 5
B. Objective Function 5
1. Algoritma Hill-Climbing 10
2. Algoritma Simulated Annealing 18
3. Algoritma Genetic 22
F. Hasil dan Analisis 31
1. Algoritma Hill-Climbing 31
2. Algoritma Simulated Annealing 57
3. Algoritma Genetic 69
KESIMPULAN DAN SARAN **111**
PEMBAGIAN TUGAS **113**
REFERENSI **114**
LAMPIRAN **115**
Form Penggunaan Generative AI 115

DESKRIPSI PERSOALAN

PT Intim Line merupakan perusahaan yang bergerak di bidang transportasi laut menggunakan kapal kargo pengangkut kendaraan. Seiring dengan diperolehnya kontrak pengiriman dari beberapa dealer kendaraan sekaligus, perusahaan perlu mengatur penempatan berbagai kendaraan pada setiap jadwal keberangkatan. Setiap kendaraan memiliki karakteristik yang berbeda, meliputi dimensi, berat, tarif pengiriman (ShippingFee), serta perkiraan waktu kedatangan (Estimated Time of Arrival atau ETA). Di sisi lain, kapal memiliki dimensi geladak dan kapasitas angkut maksimum yang membatasi jumlah kendaraan yang dapat diangkut.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Pendekatan tersebut berpotensi menyebabkan pemanfaatan ruang geladak yang kurang optimal dan pemilihan kendaraan yang tidak menghasilkan pendapatan pengiriman maksimal. Oleh karena itu, diperlukan sistem pendukung keputusan yang mampu menentukan kombinasi kendaraan yang akan diangkut sekaligus mengatur posisi dan orientasi setiap kendaraan agar kapasitas kapal dan ruang geladak dapat dimanfaatkan secara optimal.

Permasalahan ini dimodelkan sebagai masalah optimasi penempatan dua dimensi (two-dimensional packing optimization), dengan kendaraan dan geladak kapal direpresentasikan sebagai grid berbentuk persegi atau persegi panjang. Setiap kendaraan memiliki ukuran tertentu yang dapat ditempatkan dalam orientasi horizontal atau vertikal, sedangkan koordinat penempatannya dinyatakan dalam bilangan bulat. Tidak semua kendaraan harus diangkut dan kendaraan yang tidak dipilih dapat ditempatkan di luar geladak kapal. Solusi awal dibangkitkan secara acak, kemudian diperbaiki melalui mekanisme pencarian tetangga (neighbor search) yang mencakup pertukaran posisi dua kendaraan, pemindahan kendaraan ke koordinat lain, dan perubahan orientasi kendaraan.

Tujuan utama optimasi adalah memaksimalkan total ShippingFee dari seluruh kendaraan yang berhasil diangkut, dengan tetap memenuhi batasan yang telah ditentukan. Batasan tersebut meliputi tidak adanya tumpang tindih antarkendaraan, seluruh bagian kendaraan yang diangkut harus berada di dalam batas geladak kapal, serta total berat kendaraan tidak boleh melebihi kapasitas maksimum kapal. Atribut ETA juga tersedia sebagai informasi karakteristik kendaraan, meskipun tidak menjadi faktor langsung dalam fungsi objektif maupun batasan yang ditetapkan pada permasalahan ini.

Hasil akhir sistem berupa konfigurasi penempatan kendaraan yang memberikan nilai fungsi objektif terbaik berdasarkan proses pencarian yang dilakukan. Solusi tersebut mencakup informasi kendaraan yang diangkut maupun yang tidak diangkut, posisi koordinat dan orientasi setiap kendaraan, visualisasi kondisi geladak kapal, serta nilai total ShippingFee dan informasi lain yang diperlukan untuk mengevaluasi kelayakan solusi. Dengan demikian, sistem diharapkan dapat membantu tim operasional PT Intim Line mengambil keputusan

penataan kendaraan secara lebih terstruktur dan memperoleh potensi pendapatan pengiriman yang maksimal tanpa melanggar batasan operasional kapal.

PEMBAHASAN

A. Fungsi Initial State

Setiap algoritma local search memerlukan state awal sebagai titik mulai pencarian, dan strategi pembangkitannya ikut memengaruhi perilaku algoritma. Pada tugas besar ini, state awal dibangkitkan dengan pendekatan first-fit random placement melalui fungsi random_state pada berkas core/initial.py. Pendekatan ini menjamin state awal selalu valid, yaitu tidak ada kendaraan yang tumpang tindih, keluar batas geladak, atau menyebabkan total berat melebihi kapasitas kapal.

Pendekatan first-fit dipilih karena menjamin state awal yang valid sejak langkah pertama, sementara pengacakan urutan kendaraan serta pilihan posisi dan orientasi tetap mempertahankan karakter stokastik sehingga titik mulai pencarian berbeda-beda antar seed. Implementasinya juga sederhana dan efisien, sehingga tidak membebani waktu komputasi algoritma.

Logika fungsi random_state dapat diringkas sebagai berikut.

1. Urutan kendaraan diacak terlebih dahulu untuk memberikan variasi antar pembangkitan.
2. Untuk setiap kendaraan dalam urutan yang telah diacak, dilakukan pengecekan apakah penambahan berat kendaraan tersebut masih memenuhi kapasitas kapal. Jika tidak, kendaraan tersebut langsung ditetapkan berada di luar kapal (nilai None).
3. Jika kapasitas masih cukup, orientasi diacak antara H (horizontal) dan V (vertikal) untuk kendaraan yang bukan persegi.
4. Posisi (x, y) diacak dari seluruh titik pojok yang memungkinkan, dan untuk setiap posisi diperiksa apakah sel yang akan ditempati tidak tumpang tindih dengan sel yang sudah dipakai kendaraan sebelumnya. Posisi pertama yang memenuhi langsung dipilih.
5. Jika seluruh posisi dan orientasi tidak ada yang bisa menampung kendaraan tanpa pelanggaran, kendaraan ditetapkan berada di luar kapal.

B. Objective Function

Fungsi objektif merupakan komponen inti dari seluruh algoritma local search karena berfungsi sebagai pembanding kualitas antara satu solusi dengan solusi lainnya. Algoritma akan memaksimalkan nilai fungsi objektif f(s), dengan s merupakan representasi satu solusi lengkap yang berupa pemetaan setiap kendaraan ke posisi dan orientasinya di geladak, atau penanda bahwa kendaraan berada di luar kapal. Agar fungsi ini dapat digunakan oleh seluruh algoritma secara seragam, perumusan nilainya harus mencerminkan dua hal sekaligus, yaitu kualitas pendapatan tarif pengiriman dan kepatuhan terhadap hard constraint yang telah dirumuskan pada deskripsi persoalan.

Pada persoalan penataan kendaraan di geladak kapal, terdapat empat aspek yang menentukan kualitas suatu solusi, yaitu besarnya pendapatan tarif pengiriman, kepatuhan terhadap batas geladak, larangan tumpang tindih antar kendaraan, dan batas kapasitas berat kapal. Hanya aspek pertama yang hendak dimaksimalkan, sedangkan tiga aspek sisanya merupakan hard constraint yang semestinya tidak dilanggar. Agar algoritma *local search* dapat membandingkan dua solusi dari satu nilai tunggal, fungsi objektif perlu merangkum keempat aspek ini dalam satu rumus yang memberi penghargaan pada total tarif sekaligus memberi hukuman proporsional pada setiap bentuk pelanggaran.

Terdapat dua pendekatan umum untuk menangani hard constraint pada local search, yakni pendekatan larangan mutlak yang menolak setiap neighbor yang melanggar aturan, dan pendekatan penalti yang tetap mengizinkan neighbor melanggar aturan tetapi memberikan hukuman nilai besar. Pada tugas besar ini, dipilih pendekatan penalti dengan pertimbangan utama bahwa ruang solusi yang benar-benar valid pada persoalan penataan geladak cenderung sempit dan terpisah-pisah. Jika pendekatan larangan mutlak digunakan, algoritma dapat dengan mudah terjebak pada local optima karena tidak ada jalur transisi yang sah untuk berpindah antar wilayah valid. Pendekatan penalti membuka kemungkinan algoritma untuk melewati wilayah tidak valid dalam proses pencariannya, lalu kembali ke wilayah valid dengan konfigurasi yang lebih baik.

Rumus fungsi objektif yang digunakan adalah sebagai berikut.

$$f(s) = \sum_{v \in \text{inside}(s)} \text{fee}(v) - P \cdot V(s)$$

- inside(s) adalah himpunan kendaraan yang berstatus diangkut pada solusi s.
- fee(v) adalah tarif pengiriman kendaraan v.
- V(s) adalah jumlah satuan pelanggaran pada solusi s yang dirumuskan sebagai jumlah sel tumpang tindih ditambah jumlah sel yang keluar dari batas geladak ditambah kelebihan berat dalam satuan kilogram yang dibulatkan ke atas.
- P adalah koefisien penalti yang didefinisikan sebagai jumlah seluruh tarif seluruh kendaraan ditambah satu, dirumuskan sebagai berikut.

$$P = \sum_{v \in V} \text{fee}(v) + 1$$

Pemilihan koefisien penalti P sebesar total seluruh tarif ditambah satu memiliki jaminan matematis yang penting. Pada kondisi terburuk, solusi valid paling buruk memiliki nilai nol (seluruh kendaraan berada di luar kapal), sedangkan solusi tidak valid paling ringan hanya menyimpan satu satuan pelanggaran. Dengan nilai P yang lebih

besar daripada total seluruh tarif, nilai solusi tidak valid dengan satu satuan pelanggaran akan selalu lebih kecil daripada solusi valid yang mengangkut seluruh kendaraan. Jaminan ini memastikan bahwa pada akhir pencarian, solusi valid apa pun akan selalu lebih diutamakan dibanding solusi tidak valid, berapa pun banyaknya pelanggaran.

Berikut contoh perhitungan nilai fungsi objektif untuk kasus yang telah dijelaskan di atas, dengan total tarif seluruh kendaraan sebesar 92 sehingga P = 93.

State	Fee Terangkut	Pelanggaran	Perhitungan	f(s)
Solusi valid (B dan E di dalam, A, C, D di luar)	25 + 30 = 55	0	55 – 93 × 0	55
Solusi tidak valid (B, C, E, D di dalam dengan tumpang tindih dan kelebihan berat)	25 + 12 + 30 + 15 = 82	2 sel tumpang tindih + 11 kg kelebihan berat = 13	82 – 93 × 13	– 1127
Semua kendaraan di luar	0	0	0 – 93 × 0	0

Dari contoh di atas terlihat bahwa solusi tidak valid memperoleh nilai yang jauh lebih rendah daripada solusi valid dengan hasil tarif pengiriman yang lebih sedikit sekalipun. Dengan demikian, algoritma akan secara alami terdorong menuju wilayah valid tanpa perlu adanya mekanisme penolakan eksplisit terhadap neighbor yang melanggar aturan.

C. Objective Function Alternatif (Bonus)

Sebagai pelengkap, diimplementasikan juga satu fungsi objektif alternatif pada kelas Objective di src/core/objective.py yang dapat dipilih melalui parameter mode="alt" saat instansiasi, tanpa mengubah variabel yang sudah ada pada representasi state maupun pada dataset. Rumusnya adalah sebagai berikut.

$$f_{\text{alt}}(s) = \sum_{v \in \text{inside}(s)} \frac{\text{fee}(v)}{\text{area}(v)} - P \cdot V(s)$$

dengan area(v) = w(v) × l(v) merupakan luas footprint natural kendaraan. Berbeda dari fungsi objektif default yang menjumlahkan tarif total, fungsi alternatif ini mengukur efisiensi tarif per unit luas, sehingga karakter solusinya cenderung memprioritaskan kendaraan dengan rasio tarif-per-sel yang tinggi, biasanya kendaraan

berukuran kecil dengan tarif pengiriman yang mahal. Komponen penalti P · V(s) dipertahankan sama agar mekanisme kepatuhan terhadap hard constraint tidak berubah. Fungsi default tetap digunakan pada seluruh eksperimen di laporan ini, sedangkan fungsi alternatif disediakan sebagai pilihan yang tersedia untuk eksplorasi lebih lanjut.

D. Utility Function

Untuk menjaga prinsip *separation of concerns* dan menghindari duplikasi kode antar algoritma, seluruh fungsi pendukung yang bersifat umum ditempatkan pada paket core/. Paket ini berisi definisi data dasar, representasi state, pembangkit state awal, pembangkit neighbor, fungsi objektif, dan utilitas penyimpanan hasil. Setiap algoritma local search (termasuk Simulated Annealing) hanya perlu memanggil fungsi-fungsi dari paket ini tanpa perlu mendefinisikan ulang.

Berikut tabel rincian setiap kelas dan fungsi penting pada paket core/ beserta tujuan, parameter utama, dan nilai kembaliannya.

Nama	Tujuan
Vehicle (kelas)	Representasi satu kendaraan dengan atribut id, lebar, panjang, tarif pengiriman (*fee*), dan berat. Memiliki metode footprint(orientation) yang mengembalikan tuple (kolom, baris) yang ditempati pada orientasi tertentu, dan properti square yang bernilai True jika lebar sama dengan panjang (sehingga putaran tidak mengubah bentuk)
Ship (kelas)	Representasi geladak kapal dengan atribut lebar, panjang, dan kapasitas berat maksimum. Dimensi digunakan sebagai batas *grid* kolom dan baris
Problem (kelas)	Pembungkus satu kasus persoalan yang berisi objek Ship dan daftar kendaraan. Menyediakan indeks by untuk mencari kendaraan berdasarkan id dan daftar ids berisi seluruh id kendaraan
load_problem(path)	Membaca berkas JSON pada *path* yang diberikan dan mengembalikan objek Problem sesuai isinya
State (kelas)	Representasi satu susunan geladak berupa pemetaan {id_kendaraan: (x, y, orientasi)} atau None jika kendaraan di luar kapal. Menyediakan metode inside(), outside(), cells(id), weight(), fee(), dan render() untuk visualisasi teks

random_state(problem, rng)	Membangkitkan *state* awal secara acak sesuai aturan yang telah dijelaskan pada sub-bab *Pemilihan Fungsi Initial State*
random_neighbor(state, rng)	Membangkitkan satu *neighbor* acak dengan memilih satu dari tiga jenis *move*, yakni tukar dua kendaraan, pindahkan atau masukkan atau keluarkan satu kendaraan, atau putar orientasi satu kendaraan. Dipakai oleh algoritma SA dan GA
all_neighbors(state)	Mengembalikan seluruh *neighbor* yang mungkin dari *state* saat ini. Dipakai oleh algoritma *Hill-climbing* Steepest Ascent dan Sideways Move
violations(state)	Menghitung banyaknya pelanggaran dalam bentuk tuple (overlap_cells, out_of_bounds, excess_weight)
violation_units(state)	Menyatukan tiga jenis pelanggaran menjadi satu satuan pelanggaran agar dapat dikalikan dengan koefisien penalti
is_valid(state)	Mengembalikan True jika violation_units(state) sama dengan nol
Objective (kelas)	Fungsi objektif yang disiapkan sekali pada awal eksperimen. Metode value(state) menghitung nilai fee(state) - penalty * violation_units(state) sesuai rumus pada sub-bab *Pemilihan Objective Function*
make_result(name, objective, initial, final, history, iterations, duration, extra=None)	Membungkus hasil eksekusi algoritma ke dalam struktur seragam yang berisi nama algoritma, *state* awal, *state* akhir, jejak nilai objektif, jumlah iterasi, dan durasi
summary(result)	Mengembalikan ringkasan satu baris berisi nama algoritma, nilai awal, nilai akhir, validitas, iterasi, dan durasi
save_json(result, path)	Menyimpan objek hasil ke berkas JSON pada *path* yang diberikan

E. Implementasi

1. Algoritma Hill-Climbing

Hill-climbing adalah algoritma local search yang bergerak dari satu state ke state tetangga yang memiliki nilai objective function lebih tinggi. Algoritma dimulai dari sebuah initial state acak, lalu pada setiap iterasi memeriksa tetangga dari state

saat ini dan berpindah apabila ditemukan tetangga yang lebih baik. Pencarian berhenti ketika tidak ada lagi tetangga yang memberikan nilai lebih tinggi. Kondisi ini disebut local optimum, yaitu titik yang lebih baik dari seluruh tetangganya tetapi belum tentu merupakan solusi terbaik secara keseluruhan (global optimum).

Pada persoalan ini, state berupa penempatan seluruh kendaraan, yaitu posisi (x, y) dan orientasi setiap kendaraan di dalam geladak, atau keterangan bahwa kendaraan berada di luar kapal. Tetangga dari sebuah state diperoleh dengan satu langkah perubahan sesuai spesifikasi, yaitu menukar dua kendaraan, memindahkan sebuah kendaraan ke koordinat lain (termasuk memasukkan atau mengeluarkan kendaraan dari kapal), atau mengubah orientasi kendaraan. Pada dataset medium, satu state memiliki sekitar 3.000 tetangga.

Berikut ini merupakan *source code* implementasi algoritma Hill-Climbing:

```
import random
import time

from core import State, random_state, random_neighbor, all_neighbors,
make_result

def best_neighbor(state, objective):
    best, best_value = None, None
    for neighbor in all_neighbors(state):
        value = objective.value(neighbor)
        if best_value is None or value > best_value:
            best, best_value = neighbor, value
    return best, best_value


def best_neighbors(state, objective):
    best, best_value = [], None
    for neighbor in all_neighbors(state):
        value = objective.value(neighbor)
        if best_value is None or value > best_value:
            best, best_value = [neighbor], value
        elif value == best_value:
            best.append(neighbor)
    return best, best_value


def steepest_ascent(problem, objective, seed=0, max_iter=1000):
    rng = random.Random(seed)
    current = random_state(problem, rng)
    current_value = objective.value(current)
    initial = current.copy()
    objective_trace = [current_value]
    iteration = 0
    started_at = time.perf_counter()
    while iteration < max_iter:
        neighbor, neighbor_value = best_neighbor(current, objective)
        if neighbor is None or neighbor_value <= current_value:
            break
        current, current_value = neighbor, neighbor_value
        iteration += 1
        objective_trace.append(current_value)
    return make_result("steepest ascent", objective, initial, current,
objective_trace, iteration, time.perf_counter() - started_at)
```

```
def stochastic(problem, objective, seed=0, max_iter=20000, max_fail=1000):
    rng = random.Random(seed)
    current = random_state(problem, rng)
    current_value = objective.value(current)
    initial = current.copy()
    objective_trace = [current_value]
    fail_count = 0  #gagal berturut-turut
    iteration = 0
    started_at = time.perf_counter()
    while iteration < max_iter and fail_count < max_fail:
        neighbor = random_neighbor(current, rng)
        neighbor_value = objective.value(neighbor)
        if neighbor_value > current_value:
            current, current_value = neighbor, neighbor_value
            fail_count = 0
        else:
            fail_count += 1
        iteration += 1
        objective_trace.append(current_value)
    return make_result("stochastic", objective, initial, current,
objective_trace, iteration, time.perf_counter() - started_at,
        {"max_iter": max_iter, "max_fail": max_fail})


def random_restart(problem, objective, seed=0, max_restart=5,
max_iter=1000):
    objective_trace = []    #gabungan dr nilai objective semua restart
    iterations_per_restart = []
    final_per_restart = []
    best = None
    started_at = time.perf_counter()
    for restart in range(max_restart + 1):
        res = steepest_ascent(problem, objective, seed=seed + restart *
1000, max_iter=max_iter)
        if restart == 0:
            initial = State(problem, res["initial_placements"])
        objective_trace.extend(res["history"])
        iterations_per_restart.append(res["iterations"])
        final_per_restart.append(res["final_value"])
        if best is None or res["final_value"] > best["final_value"]:
            best, best_restart = res, restart
    final = State(problem, best["final_placements"])
    return make_result("random restart", objective, initial, final,
objective_trace, sum(iterations_per_restart), time.perf_counter() -
started_at,
        {"max_restart": max_restart, "iterations_per_restart":
iterations_per_restart,
         "final_per_restart": final_per_restart, "best_restart":
best_restart})


def sideways_move(problem, objective, seed=0, max_sideways=50,
max_iter=1000):
    rng = random.Random(seed)
    current = random_state(problem, rng)
    current_value = objective.value(current)
    initial = current.copy()
    objective_trace = [current_value]
    sideways_count = 0
    iteration = 0
```

```
    started_at = time.perf_counter()
    while iteration < max_iter:
        neighbors, neighbor_value = best_neighbors(current, objective)
        if not neighbors or neighbor_value < current_value:
            break
        if neighbor_value == current_value:    #langkah menyamping
            if sideways_count >= max_sideways:
                break
            sideways_count += 1
        current, current_value = rng.choice(neighbors), neighbor_value
        iteration += 1
        objective_trace.append(current_value)
    return make_result("sideways move", objective, initial, current,
objective_trace, iteration, time.perf_counter() - started_at,
        {"max_sideways": max_sideways, "sideways_count": sideways_count})
```

Berikut ini merupakan penjelasan lebih detail untuk masing-masing fungsi yang digunakan dalam implementasi algoritma hill-climbing:

Nama fungsi	def best_neighbor(state, objective)
Parameter	state (State): state saat ini objective (Objective): fungsi penilaian state
Deskripsi	Fungsi ini membangkitkan seluruh neighbor dari state menggunakan all_neighbors, menghitung nilai objektif setiap neighbor, lalu mencari satu neighbor dengan nilai objektif tertinggi. Jika terdapat beberapa neighbor dengan nilai tertinggi yang sama, neighbor yang pertama ditemukan yang dipilih. Fungsi ini digunakan oleh Steepest Ascent.
Output	Tuple (best, best_value) berisi neighbor dengan nilai objektif tertinggi beserta nilainya, atau (None, None) jika state tidak memiliki neighbor.

Source code	`def best_neighbor(state, objective):`
	`  best, best_value = None, None`
	`  for neighbor in all_neighbors(state):`
	`  value = objective.value(neighbor)`
	`  if best_value is None or value > best_value:`
	`  best, best_value = neighbor, value`
	`  return best, best_value`

Nama fungsi	def best_neighbors(state, objective):
Parameter	state (State): state saat ini objective (Objective): fungsi penilaian state
Deskripsi	Fungsi ini mirip dengan best_neighbor, tetapi

	menyimpan seluruh neighbor yang memiliki nilai objektif tertinggi yang sama, bukan hanya satu. Ketika ditemukan neighbor dengan nilai lebih tinggi, daftar dikosongkan dan diisi neighbor tersebut. Ketika ditemukan neighbor dengan nilai sama, neighbor tersebut ditambahkan ke daftar. Fungsi ini digunakan oleh Sideways Move agar langkah menyamping dapat dipilih secara acak dari beberapa pilihan, sehingga algoritma tidak bolak-balik di antara dua state yang sama.
Output	Tuple (best, best_value) berisi list neighbor dengan nilai objektif tertinggi beserta nilainya, atau ([], None) jika state tidak memiliki neighbor.

Source code	`def best_neighbors(state, objective):`
	`  best, best_value = [], None`
	`  for neighbor in all_neighbors(state):`
	`  value = objective.value(neighbor)`
	`  if best_value is None or value > best_value:`
	`  best, best_value = [neighbor], value`
	`  elif value == best_value:`
	`  best.append(neighbor)`
	`  return best, best_value`

Nama fungsi	def steepest_ascent(problem, objective, seed=0, max_iter=1000)
Parameter	problem (Problem): objek persoalan berisi kapal dan daftar kendaraan objective (Objective): fungsi penilaian state seed (int): benih acak agar initial state dapat diulang max_iter (int): batas maksimum iterasi sebagai pengaman
Deskripsi	Fungsi ini menjalankan Steepest Ascent Hill-climbing. Initial state dibangkitkan secara acak berdasarkan seed. Pada setiap iterasi, fungsi memanggil best_neighbor dan berpindah ke neighbor terbaik jika nilai objektifnya lebih tinggi dari state saat ini. Pencarian berhenti ketika neighbor terbaik tidak lebih baik dari state saat ini (local optimum) atau jumlah iterasi mencapai max_iter. Satu iterasi dihitung sebagai satu kali perpindahan state. Nilai objektif setiap iterasi disimpan dalam objective_trace untuk keperluan plot.

Output	Objek hasil dari make_result berisi nama algoritma, nilai objektif awal dan akhir, status validitas state akhir, riwayat nilai objektif per iterasi, banyak iterasi, durasi pencarian, serta posisi kendaraan pada state awal dan akhir.

Source code	`def steepest_ascent(problem, objective, seed=0, max_iter=1000):`
	`  rng = random.Random(seed)`
	`  current = random_state(problem, rng)`
	`  current_value = objective.value(current)`
	`  initial = current.copy()`
	`  objective_trace = [current_value]`
	`  iteration = 0`
	`  started_at = time.perf_counter()`
	`  while iteration < max_iter:`
	`  neighbor, neighbor_value = best_neighbor(current, objective)`
	`  if neighbor is None or neighbor_value <= current_value:`
	`  break`
	`  current, current_value = neighbor, neighbor_value`
	`  iteration += 1`
	`  objective_trace.append(current_value)`
	`  return make_result("steepest ascent", objective, initial, current, objective_trace, iteration, time.perf_counter() - started_at)`

Nama fungsi	def stochastic(problem, objective, seed=0, max_iter=20000, max_fail=1000)
Parameter	problem (Problem): objek persoalan berisi kapal dan daftar kendaraan objective (Objective): fungsi penilaian state seed (int): benih acak agar hasil dapat diulang max_iter (int): batas maksimum iterasi max_fail (int): batas kegagalan berturut-turut sebelum pencarian dihentikan
Deskripsi	Fungsi ini menjalankan Stochastic Hill-climbing. Pada setiap iterasi, satu neighbor dibangkitkan secara acak menggunakan random_neighbor. Jika nilai objektifnya lebih tinggi, algoritma berpindah ke neighbor tersebut dan fail_count dikembalikan ke 0. Jika tidak, fail_count bertambah satu. Pencarian berhenti ketika fail_count mencapai max_fail atau jumlah iterasi mencapai max_iter. Satu iterasi dihitung sebagai satu kali percobaan neighbor, termasuk percobaan yang gagal.

	Nilai max_fail = 1000 dipilih berdasarkan uji coba: dengan nilai 100, sebagian hasil akhir masih melanggar batasan, sedangkan mulai nilai 500 hasil akhir tidak berubah lagi.
Output	Objek hasil dari make_result dengan isi yang sama seperti steepest_ascent, ditambah data extra berupa nilai max_iter dan max_fail yang digunakan.

Source code	`def stochastic(problem, objective, seed=0, max_iter=20000, max_fail=1000):`
	`  rng = random.Random(seed)`
	`  current = random_state(problem, rng)`
	`  current_value = objective.value(current)`
	`  initial = current.copy()`
	`  objective_trace = [current_value]`
	`  fail_count = 0  #gagal berturut-turut`
	`  iteration = 0`
	`  started_at = time.perf_counter()`
	`  while iteration < max_iter and fail_count < max_fail:`
	`  neighbor = random_neighbor(current, rng)`
	`  neighbor_value = objective.value(neighbor)`
	`  if neighbor_value > current_value:`
	`  current, current_value = neighbor, neighbor_value`
	`  fail_count = 0`
	`  else:`
	`  fail_count += 1`
	`  iteration += 1`
	`  objective_trace.append(current_value)`
	`  return make_result("stochastic", objective, initial, current, objective_trace, iteration, time.perf_counter() - started_at,`
	`  {"max_iter": max_iter, "max_fail": max_fail})`

Nama fungsi	def random_restart(problem, objective, seed=0, max_restart=5, max_iter=1000)
Parameter	problem (Problem): objek persoalan berisi kapal dan daftar kendaraan objective (Objective): fungsi penilaian state seed (int): benih acak untuk percobaan pertama max_restart (int): jumlah maksimum restart max_iter (int): batas maksimum iterasi setiap pemanggilan Steepest Ascent
Deskripsi	Fungsi ini menjalankan Random Restart Hill-climbing dengan memanggil steepest_ascent sebanyak max_restart + 1 kali, yaitu satu percobaan awal ditambah max_restart kali restart. Percobaan pertama

	menggunakan seed yang sama dengan varian lain sehingga initial state-nya sama, sedangkan percobaan berikutnya menggunakan seed + restart * 1000 agar initial state berbeda. Riwayat nilai objektif setiap percobaan digabung menjadi satu, banyak iterasi setiap percobaan dicatat, dan hasil dengan nilai akhir tertinggi disimpan sebagai solusi. State awal dan akhir dibentuk ulang dari data posisi kendaraan menggunakan State.
Output	Objek hasil dari make_result dengan isi yang sama seperti steepest_ascent, ditambah data extra berupa nilai max_restart dan daftar banyak iterasi per restart (iterations_per_restart). Banyak iterasi total merupakan jumlah iterasi seluruh percobaan.

Source code	`def random_restart(problem, objective, seed=0, max_restart=5, max_iter=1000):`
	`  objective_trace = []`
	`  iterations_per_restart = []`
	`  best = None`
	`  started_at = time.perf_counter()`
	`  for restart in range(max_restart + 1):`
	`  res = steepest_ascent(problem, objective, seed=seed + restart * 1000, max_iter=max_iter)`
	`  if restart == 0:`
	`  initial = State(problem, res["initial_placements"])`
	`  objective_trace.extend(res["history"])`
	
	`iterations_per_restart.append(res["iterations"])`
	`  if best is None or res["final_value"] > best["final_value"]:`
	`  best = res`
	`  final = State(problem, best["final_placements"])`
	`  return make_result("random restart", objective, initial, final, objective_trace, sum(iterations_per_restart), time.perf_counter() - started_at,`
	`  {"max_restart": max_restart, "iterations_per_restart": iterations_per_restart})`

Nama fungsi	def sideways_move(problem, objective, seed=0, max_sideways=50, max_iter=1000)
Parameter	problem (Problem): objek persoalan berisi kapal dan daftar kendaraan. objective (Objective): fungsi penilaian state. seed (int): benih acak agar hasil dapat diulang. max_sideways (int): jumlah maksimum langkah

	menyamping. max_iter (int): batas maksimum iterasi sebagai pengaman
Deskripsi	Fungsi ini menjalankan Hill-climbing with Sideways Move. Pada setiap iterasi, fungsi memanggil best_neighbors. Jika nilai objektif neighbor terbaik lebih tinggi, algoritma berpindah seperti Steepest Ascent. Jika nilainya sama, algoritma tetap berpindah ke salah satu neighbor terbaik yang dipilih secara acak dan sideways_count bertambah satu. Pencarian berhenti ketika neighbor terbaik lebih buruk dari state saat ini, ketika sideways_count mencapai max_sideways, atau ketika jumlah iterasi mencapai max_iter. sideways_count menghitung total langkah menyamping selama pencarian, sesuai ketentuan spesifikasi bahwa pencarian dihentikan ketika banyak sideways move mencapai batas maksimum.
Output	Objek hasil dari make_result dengan isi yang sama seperti steepest_ascent, ditambah data extra berupa nilai max_sideways dan jumlah langkah menyamping yang dilakukan (sideways_count).

Source code	`def sideways_move(problem, objective, seed=0, max_sideways=50, max_iter=1000):`
	`  rng = random.Random(seed)`
	`  current = random_state(problem, rng)`
	`  current_value = objective.value(current)`
	`  initial = current.copy()`
	`  objective_trace = [current_value]`
	`  sideways_count = 0`
	`  iteration = 0`
	`  started_at = time.perf_counter()`
	`  while iteration < max_iter:`
	`  neighbors, neighbor_value = best_neighbors(current, objective)`
	`  if not neighbors or neighbor_value < current_value:`
	`  break`
	`  if neighbor_value == current_value: #langkah menyamping`
	`  if sideways_count >= max_sideways:`
	`  break`
	`  sideways_count += 1`
	`  current, current_value = rng.choice(neighbors), neighbor_value`
	`  iteration += 1`
	`  objective_trace.append(current_value)`
	`  return make_result("sideways move", objective, initial, current, objective_trace, iteration, time.perf_counter() - started_at,`
	`  {"max_sideways": max_sideways,`

	"sideways_count": sideways_count})

2. Algoritma Simulated Annealing

Simulated Annealing (SA) merupakan algoritma pencarian lokal yang mengambil inspirasi dari proses pendinginan logam di bidang metalurgi. Perbedaan utama antara SA dengan Hill-climbing terletak pada perlakuan terhadap langkah memburuk. Jika Hill-climbing hanya menerima langkah yang menaikkan nilai objektif, SA memberi ruang bagi langkah memburuk untuk diterima dengan peluang tertentu yang dikendalikan oleh parameter suhu. Mekanisme ini memungkinkan SA untuk keluar dari local optima dan menjangkau wilayah ruang solusi yang lebih luas, kemampuan yang tidak dimiliki Hill-climbing biasa.

Pada SA, suhu berperan sebagai pengatur keseimbangan antara eksplorasi dan eksploitasi. Pada fase awal pencarian ketika suhu masih tinggi, peluang menerima langkah memburuk juga tinggi sehingga algoritma dapat menjelajah ruang solusi secara leluasa. Seiring iterasi berjalan, suhu diturunkan secara bertahap melalui jadwal pendinginan, dan peluang penerimaan langkah memburuk pun menurun. Pada fase akhir, SA menjadi lebih selektif dan karakternya mendekati Hill-climbing yang berfokus menyempurnakan solusi pada area yang telah dianggap menjanjikan. Dengan pola tersebut, SA memiliki peluang lebih besar untuk mencapai global optimum, terutama pada persoalan dengan ruang solusi besar dan banyak local optima.

Berikut ini merupakan *source code* implementasi algoritma simulated annealing:

```
import math
import random
import time

from core import random_state, random_neighbor, make_result

def boltzmann_gate(delta_e, temp):
    """Peluang penerimaan langkah memburuk"""
    if temp <= 0:
        return 0.0
    return math.exp(delta_e / temp)


def cooldown_geometric(temp, alpha):
    """Jadwal pendinginan geometrik/suhu berikutnya"""
    return temp * alpha

def simulated_annealing(problem, objective, seed=0, t0=100.0, alpha=0.999,
t_min=1e-3, max_iter=30000, stuck_window=200):
    rng = random.Random(seed)
    current = random_state(problem, rng)
    current_value = objective.value(current)
    initial = current.copy()    # arsip kondisi awal
```

```
    objective_trace = [current_value]    # nilai objective per iterasi
    accept_trace = []    # nilai e^(dE/T) per iterasi
    temp_trace = []    # suhu per iterasi
    stuck_count = 0    # kejadian tertahan di local optimum
    idle_steps = 0    # jumlah iterasi tanpa perbaikan

    temp = t0
    iteration = 0
    started_at = time.perf_counter()
    while iteration < max_iter and temp > t_min:
        iteration += 1
        candidate = random_neighbor(current, rng)
        candidate_value = objective.value(candidate)
        delta_e = candidate_value - current_value

        if delta_e > 0:
            accept_trace.append(1.0)    # langkah membaik selalu diterima
            current, current_value = candidate, candidate_value
            idle_steps = 0    # reset idle steps karena ada perbaikan
        else:
            prob = boltzmann_gate(delta_e, temp)
            accept_trace.append(prob)
            if prob > rng.random():    # kriteria metropolis
                current, current_value = candidate, candidate_value
            idle_steps += 1

        temp_trace.append(temp)
        objective_trace.append(current_value)
        if idle_steps >= stuck_window:    # penentuan stuck
            stuck_count += 1
            idle_steps = 0    # reset idle steps karena sudah tercatat
sebagai stuck
        temp = cooldown_geometric(temp, alpha)    # turunkan suhu tiap
iterasi

    return make_result("Simulated Annealing", objective, initial, current,
objective_trace, iteration, time.perf_counter() - started_at,
        {"acceptance_probs": accept_trace, "temperatures": temp_trace,
"stuck_count": stuck_count},)
```

Berikut ini merupakan penjelasan lebih detail untuk masing-masing fungsi yang digunakan dalam implementasi algoritma Simulated Annealing:

Nama fungsi	def boltzmann_gate(delta_e, temp)
Parameter	delta_e (float): selisih nilai objektif antara neighbor dengan state saat ini. temp (float): suhu pada iterasi saat ini.
Deskripsi	Fungsi ini menghitung peluang penerimaan langkah memburuk berdasarkan distribusi Boltzmann dengan rumus $e^{dE/T}$. Apabila suhu bernilai nol atau negatif, fungsi mengembalikan nol untuk mencegah pembagian oleh nol

Output	Float pada rentang 0,0 sampai 1,0 yang menyatakan peluang menerima neighbor yang menurunkan nilai objektif

Source code	`def boltzmann_gate(delta_e, temp):`
	`    """Peluang penerimaan langkah memburuk"""`
	`    if temp <= 0:`
	`        return 0.0`
	`    return math.exp(delta_e / temp)`

Nama fungsi	def cooldown_geometric(temp, alpha)
Parameter	temp (float): suhu saat ini. alpha (float, rentang 0 sampai 1): koefisien pendinginan geometrik.
Deskripsi	Fungsi ini menghitung suhu untuk iterasi berikutnya dengan mengalikan suhu saat ini dengan koefisien alpha. Karena alpha selalu kurang dari satu, suhu menurun secara eksponensial seiring iterasi
Output	Float suhu baru yang digunakan pada iterasi berikutnya

Source code	`def cooldown_geometric(temp, alpha):`
	`    """Jadwal pendinginan geometrik/suhu berikutnya"""`
	`    return temp * alpha`

Nama fungsi	def simulated_annealing(problem, objective, seed=0, t0=100.0, alpha=0.999, t_min=1e-3, max_iter=30000, stuck_window=200)
Parameter	problem (Problem): objek persoalan berisi kapal dan daftar kendaraan. objective (Objective): fungsi penilaian. seed (int): benih acak. t0 (float): suhu awal. alpha (float): koefisien pendinginan. t_min (float): ambang suhu minimum. max_iter (int): batas iterasi maksimum. stuck_window (int): jendela iterasi tanpa peningkatan untuk mencatat kejadian stuck.
Deskripsi	Fungsi ini menjalankan algoritma Simulated Annealing dengan aturan penerimaan Metropolis. Fungsi membangkitkan state awal valid lewat first-fit, lalu secara iteratif memilih neighbor acak dan memutuskan

	menerima atau menolak berdasarkan selisih nilai objektif dan suhu saat ini. Suhu diturunkan setiap iterasi dengan jadwal pendinginan geometrik. Loop berhenti ketika suhu turun di bawah t_min atau jumlah iterasi mencapai max_iter
Output	Objek hasil dari make_result berisi nama algoritma, nilai awal dan akhir, status validitas, jejak nilai objektif, jumlah iterasi, durasi eksekusi, serta tracking tambahan berupa acceptance_probs (daftar nilai peluang penerimaan $e^{dE/T}$ tiap iterasi, dipakai untuk plot peluang penerimaan vs iterasi), temperatures (daftar nilai suhu tiap iterasi, dipakai untuk plot dinamika pendinginan jika diperlukan), dan stuck_count (jumlah kejadian stuck di local optima, yakni berapa kali algoritma mengalami rentetan 200 iterasi berturut-turut tanpa peningkatan nilai current)

Source code	`def simulated_annealing(problem, objective, seed=0, t0=100.0, alpha=0.999, t_min=1e-3, max_iter=30000, stuck_window=200):`
	`    rng = random.Random(seed)`
	`    current = random_state(problem, rng)`
	`    current_value = objective.value(current)`
	`    initial = current.copy()    # arsip kondisi awal`
	`    objective_trace = [current_value]    # nilai objective per iterasi`
	`    accept_trace = []    # nilai e^(dE/T) per iterasi`
	`    temp_trace = []    # suhu per iterasi`
	`    stuck_count = 0    # kejadian tertahan di local optimum`
	`    idle_steps = 0    # jumlah iterasi tanpa perbaikan`
	
	`    temp = t0`
	`    iteration = 0`
	`    started_at = time.perf_counter()`
	`    while iteration < max_iter and temp > t_min:`
	`        iteration += 1`
	`        candidate = random_neighbor(current, rng)`
	`        candidate_value = objective.value(candidate)`
	`        delta_e = candidate_value - current_value`
	
	`        if delta_e > 0:`
	`            accept_trace.append(1.0)    # langkah membaik selalu diterima`
	`            current, current_value = candidate, candidate_value`
	`            idle_steps = 0    # reset idle steps karena ada perbaikan`
	`        else:`
	`            prob = boltzmann_gate(delta_e, temp)`

```
            accept_trace.append(prob)
            if prob > rng.random():    # kriteria
metropolis
                current, current_value =
candidate, candidate_value
            idle_steps += 1

        temp_trace.append(temp)
        objective_trace.append(current_value)
        if idle_steps >= stuck_window:    #
penentuan stuck
            stuck_count += 1
            idle_steps = 0    # reset idle steps
karena sudah tercatat sebagai stuck
        temp = cooldown_geometric(temp, alpha)
# turunkan suhu tiap iterasi

    return make_result("Simulated Annealing",
objective, initial, current, objective_trace,
iteration, time.perf_counter() - started_at,
        {"acceptance_probs": accept_trace,
"temperatures": temp_trace, "stuck_count":
stuck_count},)
```

3. Algoritma Genetic

Genetic Algorithm merupakan algoritma optimasi yang memanfaatkan teori evolusi dalam menemukan solusi terbaik dari suatu permasalahan. Algoritma ini bekerja dengan membentuk sekumpulan individu secara acak sebagai solusi awal yang disebut populasi. Kemudian, setiap individu yang terbentuk dievaluasi menggunakan *fitness function* untuk menentukan kualitasnya. Dalam kasus ini, evaluasi mempertimbangkan total tarif pengiriman dari kendaraan di dalam kapal serta penalti apabila melanggar batasan yang telah ditentukan. Individu dengan nilai *fitness* yang lebih tinggi menunjukkan hasil optimasi yang lebih baik dan memiliki peluang besar untuk menjadi *parent*. *Parent* yang dipilih tadi akan dikombinasikan melalui *crossover* dan dimodifikasi melalui *mutation* untuk menghasilkan generasi inidividu baru. Individu terbaik yang terbentuk akan selalu dipertahankan ke generasi berikutnya untuk menjaga kualitas solusi. Proses ini dilakukan berulang kali dalam beberapa generasi hingga mencapai jumlah iterasi maksimal dengan tujuan memperoleh solusi optimal atau mendekati optimal.

Berbeda dengan *hill-climbing* dan *simulated annealing* yang memperbaiki satu solusi secara bertahap, *Genetic Algorithm* mengelola sekumpulan solusi sekaligus. Melalui *crossover*, *Genetic Algorithm* menggabungkan karakteristik dari kedua *parent* yang telah dipilih, sedangkan *mutation* memungkinkan perubahan pada individu untuk mengeksplorasi kemungkinan solusi lain. Namun, proses tersebut tidak menjamin bahwa individu baru yang terbentuk selalu lebih baik daripada individu di generasi sebelumnya.

Berikut ini merupakan *source code* implementasi *Genetic Algorithm*:

```
import random
import time
from core import (Problem, State, Objective, random_state,
random_neighbor, make_result)

class GeneticAlgorithm():
    def __init__(self, problem: Problem, population: int, iteration: int,
seed: int, prob_mutation: float = 0.1):
        self.problem = problem
        self.population = population        #jumlah individu di setiap
generasi
        self.iteration = iteration          #jumlah iterasi yang
dijalankan
        self.seed = seed                    #nilai acak agar hasil bisa
diulang
        self.prob_mutation = prob_mutation  #peluang mutasi anak
        self.rng = random.Random(seed)
        self.objective = Objective(problem)

    #Fitness function
    def fitness_function(self, state):
            """Menghitung nilai fitness untuk mengevaluasi kualitas
solusi"""
            return self.objective.value(state)

    #Initial population
    def create_initial_population(self):
        """Membuat state acak sebagai populasi awal"""
        population = []
        for _ in range(self.population):
            population.append(random_state(self.problem, self.rng))
        return population

    #Selection function
    def selection(self, population, fitness):
        """Memilih individu sebagai parent menggunakan Tournament
Selection(mengambil 3 individu acak dan memilih individu dengan nilai
fitness tertinggi)"""
        selected = []
        tournament_size = min(3, len(population))
        for _ in range (self.population):
            idx = self.rng.sample(range(len(population)), tournament_size)
            winner = max(idx, key=lambda i: fitness[i])
            selected.append(population[winner])
        return selected

    #Crossover function
    def crossover(self, parent1, parent2):
        """Melakukan uniform crossover: memilih induk secara acak"""
        ship = self.problem.ship
        children = []
        for p1, p2 in ((parent1, parent2), (parent2, parent1)):
            child = {vehicle_id: None for vehicle_id in self.problem.ids}
            occupied = set()
            weight = 0
            order = list(self.problem.ids)
            self.rng.shuffle(order)
            for vehicle_id in order:
                parents = [p1,p2]
                if self.rng.random() < 0.5:
```

```
                    parents.reverse()
                vehicle = self.problem.by[vehicle_id]
                for parent in parents:
                    if parent.p[vehicle_id] is None:
                        continue
                    cells = parent.cells(vehicle_id)
                    if (all(0 <= x < ship.cols and 0 <= y < ship.rows for
x, y in cells)
                        and weight + vehicle.weight <= ship.cap
                        and occupied.isdisjoint(cells)):
                        child[vehicle_id] = parent.p[vehicle_id]
                        occupied.update(cells)
                        weight += vehicle.weight
                        break
            children.append(State(self.problem, child))
        return children[0], children[1]

    #Mutation function
    def mutation(self, state: State):
        """Melakukan mutasi dengan swap kendaraan secara random"""
        if self.rng.random() < self.prob_mutation:
            return random_neighbor(state, self.rng)
        return state

    def get_best(self, population, fitness):
        """Individu terbaik dari populasi"""
        best = max(range(len(population)), key=lambda i: fitness[i])
        return population[best], fitness[best]

    #Main loop
    def run(self):
        self.rng = random.Random(self.seed)
        start = time.time()

        #Buat populasi
        population = self.create_initial_population()
        fitness = [self.fitness_function(state) for state in population]

        #State awal
        initial_idx = max(range(self.population), key=lambda i:
fitness[i])
        initial_state, initial_fitness = self.get_best(population,
fitness)
        initial_state = initial_state.copy()
        global_best_state = initial_state.copy()
        global_best_fitness = initial_fitness

        max_history = [global_best_fitness]
        average_history = [sum(fitness)/len(fitness)]

        #Generation Loop
        for _ in range (self.iteration):
            selected = self.selection(population, fitness)
            next_population = []
            for i in range (0, self.population, 2):
                parent1 = selected[i]
                parent2 = selected[(i + 1) % self.population]
                child1, child2 = self.crossover(parent1, parent2)
                child1 = self.mutation(child1)
                child2 = self.mutation(child2)
                next_population.append(child1)
                next_population.append(child2)
```

```
            next_population = next_population[:self.population]

            next_population[0] = global_best_state.copy()
            population = next_population
            fitness = [self.fitness_function(state) for state in
population]

            generation_best_state, generation_best_fitness =
self.get_best(population, fitness)
            if generation_best_fitness > global_best_fitness:
                global_best_state = generation_best_state.copy()
                global_best_fitness = generation_best_fitness

            max_history.append(global_best_fitness)
            average_history.append(sum(fitness)/len(fitness))

        #State akhir
        final_state = global_best_state
        duration = time.time() - start

        return make_result("Genetic Algorithm", self.objective,
initial_state, final_state, history=max_history,
iterations=self.iteration, duration=duration,
        extra={"population": self.population, "seed": self.seed,
        "max_history": max_history, "average_history": average_history})
```

Berikut ini merupakan penjelasan lebih detail untuk masing-masing fungsi yang digunakan dalam implementasi algoritma genetic:

Nama fungsi	def __init__(self, problem: Problem, population: int, iteration: int, seed: int, prob_mutation: float = 0.1)
Deskripsi	Fungsi ini merupakan konstruktor yang menginisialisasi parameter yang digunakan dalam menjalankan *Genetic Algorithm*. Pada fungsi ini juga diinisialisasi *random number generator* berdasarkan *seed* agar hasil eksperimen dapat diulang. Selain itu, diinisialisasi juga Objective untuk mengevaluasi kualitas solusi. Peluang mutasi ditetapkan sebesar 0.1 per anak untuk menjaga keragaman populasi. Nilai ini dibuat kecil agar mutasi tidak merusak individu yang sudah baik atau menghasilkan pelanggaran *constraints* yang menyebabkan penalti lebih besar.
Output	Menginisialisasi atribut yang digunakan di kelas GeneticAlgorithm.

Source code	`def __init__(self, problem: Problem, population: int, iteration: int, seed: int, prob_mutation: float = 0.1):`
	`  self.problem = problem`
	`  self.population = population`
	`#jumlah individu di setiap generasi`
	`  self.iteration = iteration`

	`#jumlah iterasi yang dijalankan`
	`self.seed = seed`
	`#nilai acak agar hasil bisa diulang`
	`self.prob_mutation = prob_mutation`
	`#peluang mutasi anak`
	`self.rng = random.Random(seed)`
	`self.objective = Objective(problem)`

Nama fungsi	def fitness_function(self, state)
Deskripsi	Fungsi ini digunakan untuk menghitung nilai fitness atau objective function dari setiap individu. Nilai ini dihitung berdasarkan total ShippingFee dikurangi dengan penalti yang terjadi akibat overlapping, keluar batas, dan melebihi kapasitas kapal. Nilai fitness ini digunakan untuk mengevaluasi kualitas setiap individu (semakin besar nilai fitness, semakin baik solusinya).
Output	Nilai fitness dari suatu individu bertipe float.

Source code	`def fitness_function(self, state):`
	`"""Menghitung nilai fitness untuk mengevaluasi kualitas solusi"""`
	`return self.objective.value(state)`

Nama fungsi	def create_initial_population(self)
Deskripsi	Fungsi ini digunakan untuk membuat populasi awal yang terdiri dari sejumlah individu sebanyak parameter population yang ditentukan. Setiap individu dibentuk secara acak berupa state posisi kendaraan di dalam dan luar kapal. Posisi tersebut tidak dijamin valid, tetapi pelanggaran *constraints* akan ditangani oleh fitness function. Sehingga populasi tersebut dapat dikembangkan melalui algoritma.
Output	List berisi beberapa objek State sebagai populasi awal.

Source code	`def create_initial_population(self):`
	`"""Membuat state acak sebagai populasi awal"""`
	`population = []`
	`for _ in range(self.population):`
	`population.append(random_state(self.problem, self.rng))`
	`return population`

Nama fungsi	def selection(self, population, fitness)

Deskripsi	Fungsi ini digunakan untuk memilih individu yang akan menjadi parent. Pemilihan dilakukan dengan metode Tournament Selection. Dalam setiap turnamen, dipilih 3 individu secara acak dan dipilih individu dengan nilai fitness tertinggi sebagai pemenang. Turnamen diulang sebanyak jumlah populasi, sehingga individu yang sama dapat terpilih lebih dari sekali. Metode Tournament Selection ini dipilih karena nilai fitness pada kasus ini cenderung negatif akibat penalti. Tournament Selection hanya membandingkan nilai fitness antar individu, sehingga tidak terpengaruh oleh nilai negatif dan tetap memberi peluang bagi individu dengan nilai fitness lebih rendah untuk terpilih.
Output	List berisi individu-individu terpilih yang berperan sebagai parent.

Source code	`def selection(self, population, fitness):`
	`  """Memilih individu sebagai parent menggunakan Tournament Selection(mengambil 3 individu acak dan memilih individu dengan nilai fitness tertinggi)"""`
	`  selected = []`
	`  tournament_size = min(3, len(population))`
	`  for _ in range (self.population):`
	`  idx = self.rng.sample(range(len(population)), tournament_size)`
	`  winner = max(idx, key=lambda i: fitness[i])`
	`  selected.append(population[winner])`
	`  return selected`

Nama fungsi	def crossover(self, parent1, parent2)
Deskripsi	Fungsi ini digunakan untuk menghasilkan dua individu baru dari parent yang terbentuk di fungsi sebelumnya. Fungsi ini menggunakan metode Uniform Crossover dengan melakukan kombinasi penempatan kendaraan dari kedua parent secara acak. Penempatan hanya dipakai apabila tidak melanggar constraints. Apabila kedua parent melanggar, kendaraan tersebut ditempatkan di luar kapal. Dengan begitu, anak yang dihasilkan selalu valid. Uniform Crossover dipilih karena setiap kendaraan dapat diwariskan secara independen tanpa bergantung pada urutan kendaraan.
Output	Dua individu baru hasil dari Uniform Crossover.

Source code	`def crossover(self, parent1, parent2):`
	`  """Melakukan uniform crossover: memilih induk secara acak"""`
	`  ship = self.problem.ship`
	`  children = []`
	`  for p1, p2 in ((parent1, parent2), (parent2, parent1)):`
	`  child = {vehicle_id: None for vehicle_id in self.problem.ids}`
	`  occupied = set()`
	`  weight = 0`
	`  order = list(self.problem.ids)`
	`  self.rng.shuffle(order)`
	`  for vehicle_id in order:`
	`  parents = [p1,p2]`
	`  if self.rng.random() < 0.5:`
	`  parents.reverse()`
	`  vehicle = self.problem.by[vehicle_id]`
	`  for parent in parents:`
	`  if parent.p[vehicle_id] is None:`
	`  continue`
	`  cells = parent.cells(vehicle_id)`
	`  if (all(0 <= x < ship.cols and 0 <= y < ship.rows for x, y in cells)`
	`  and weight + vehicle.weight <= ship.cap`
	`  and occupied.isdisjoint(cells)):`
	`  child[vehicle_id] = parent.p[vehicle_id]`
	`  occupied.update(cells)`
	`  weight += vehicle.weight`
	`  break`
	`  children.append(State(self.problem, child))`
	`  return children[0], children[1]`

Nama fungsi	def mutation(self, state: State)
Deskripsi	Fungsi ini digunakan untuk melakukan mutasi pada individu hasil crossover untuk menjaga keragaman populasi. Mutasi hanya dilakukan jika nilai random lebih kecil dari probabilitas mutasi yang telah ditentukan (0.1). Jika mutasi terjadi, individu diubah melalui fungsi random_neighbor dengan mengubah penempatan, menukar, menambah, mengeluarkan, atau memutar orientasi kendaraan. Jika tidak terjadi mutasi, individu tetap dipertahankan tanpa perubahan.
Output	Objek State hasil mutasi jika probabilitas mutasi terpenuhi atau objek State yang sama jika mutasi tidak dilakukan.

Source code	`def mutation(self, state: State):`
	`  """Melakukan mutasi dengan swap kendaraan secara random"""`
	`  if self.rng.random() < self.prob_mutation:`
	`  return random_neighbor(state, self.rng)`
	`  return state`

Nama fungsi	def get_best(self, population, fitness)
Deskripsi	Fungsi ini digunakan untuk mencari individu dengan nilai fitness tertinggi dari suatu populasi.
Output	Individu terbaik beserta nilai fitness tertingginya.

Source code	`def get_best(self, population, fitness):`
	`  """Individu terbaik dari populasi"""`
	`  best = max(range(len(population)), key=lambda i: fitness[i])`
	`  return population[best], fitness[best]`

Nama fungsi	def run(self)
Deskripsi	Fungsi ini merupakan fungsi utama yang digunakan untuk menjalankan seluruh proses Genetic Algorithm, mulai dari inisialisasi, pembentukan populasi awal, evaluasi fitness score, seleksi parent, crossover, mutasi, hingga membentuk generasi baru. Pada populasi generasi baru, individu terbaik dari generasi sebelumnya selalu dipertahankan pada posisi pertama (elitisme) agar kualitas solusi tidak menurun. Selanjutnya, *fitness* dari populasi baru dievaluasi dan solusi terbaik diperbarui jika ditemukan individu yang lebih baik. Nilai *fitness* maksimum dan rata-rata populasi dicatat pada setiap generasi untuk keperluan plot. Proses berhenti setelah jumlah iterasi yang ditentukan tercapai dengan solusi terbaik yang ditemukan menjadi *state* akhir.
Output	Solusi terbaik yang ditemukan dari Genetic Algorithm.

Source code	`def run(self):`
	`  self.rng = random.Random(self.seed)`
	`  start = time.time()`
	
	`  #Buat populasi`
	`  population = self.create_initial_population()`
	`  fitness = [self.fitness_function(state)`

```
for state in population]

        #State awal
        initial_idx = max(range(self.population),
key=lambda i: fitness[i])
        initial_state, initial_fitness =
self.get_best(population, fitness)
        initial_state = initial_state.copy()
        global_best_state = initial_state.copy()
        global_best_fitness = initial_fitness

        max_history = [global_best_fitness]
        average_history =
[sum(fitness)/len(fitness)]

        #Generation Loop
        for _ in range (self.iteration):
            selected = self.selection(population,
fitness)
            next_population = []
            for i in range (0, self.population,
2):
                parent1 = selected[i]
                parent2 = selected[(i + 1) %
self.population]
                child1, child2 =
self.crossover(parent1, parent2)
                child1 = self.mutation(child1)
                child2 = self.mutation(child2)
                next_population.append(child1)
                next_population.append(child2)
            next_population =
next_population[:self.population]

            next_population[0] =
global_best_state.copy()
            population = next_population
            fitness =
[self.fitness_function(state) for state in
population]

            generation_best_state,
generation_best_fitness =
self.get_best(population, fitness)
            if generation_best_fitness >
global_best_fitness:
                global_best_state =
generation_best_state.copy()
                global_best_fitness =
generation_best_fitness


max_history.append(global_best_fitness)

average_history.append(sum(fitness)/len(fitness))

        #State akhir
        final_state = global_best_state
        duration = time.time() - start

        return make_result("Genetic Algorithm",
self.objective, initial_state, final_state,
```

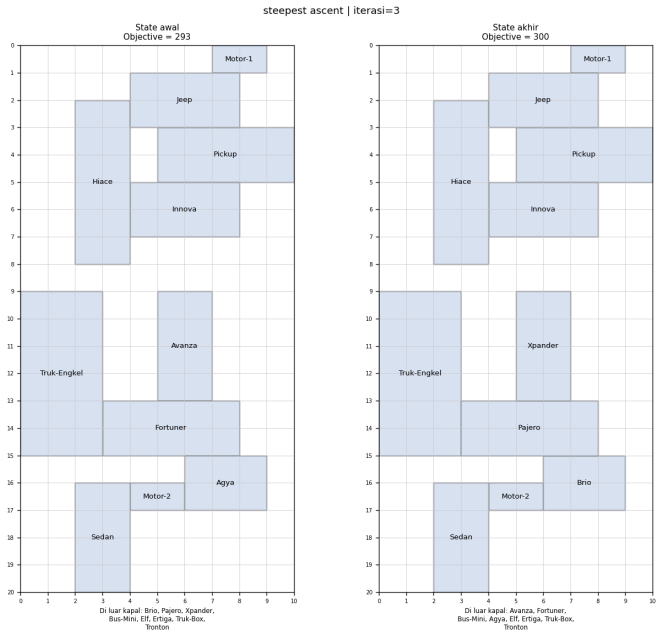
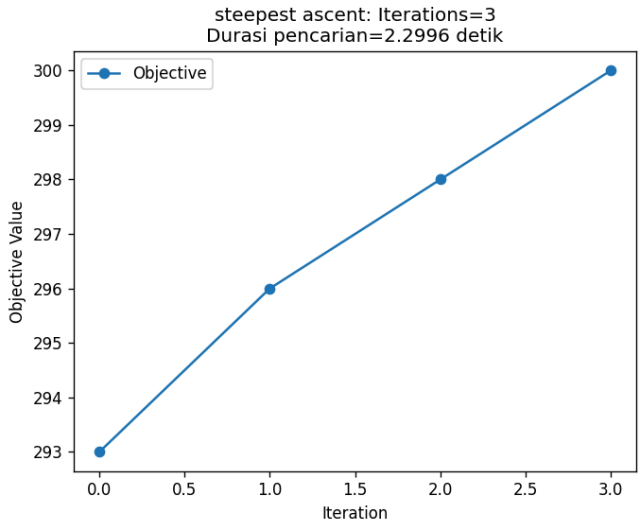
	`history=max_history, iterations=self.iteration, duration=duration, extra={"population": self.population, "seed": self.seed, "max_history": max_history, "average_history": average_history})`

F. Hasil dan Analisis

1. Algoritma Hill-Climbing

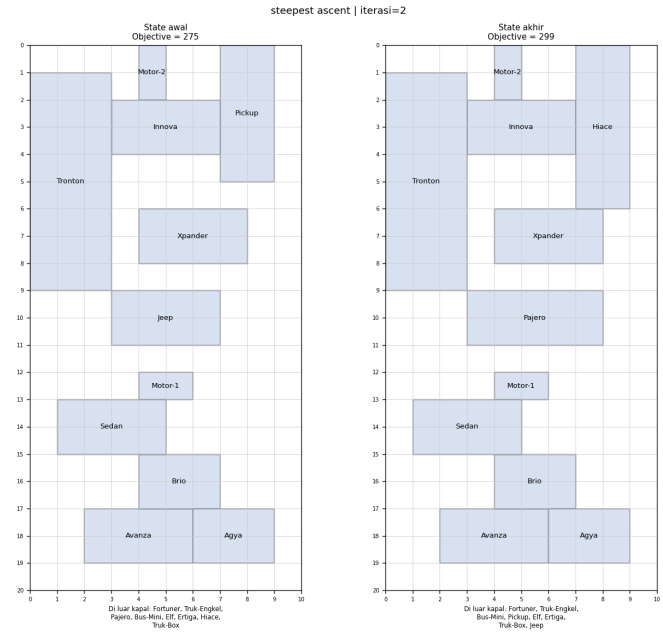
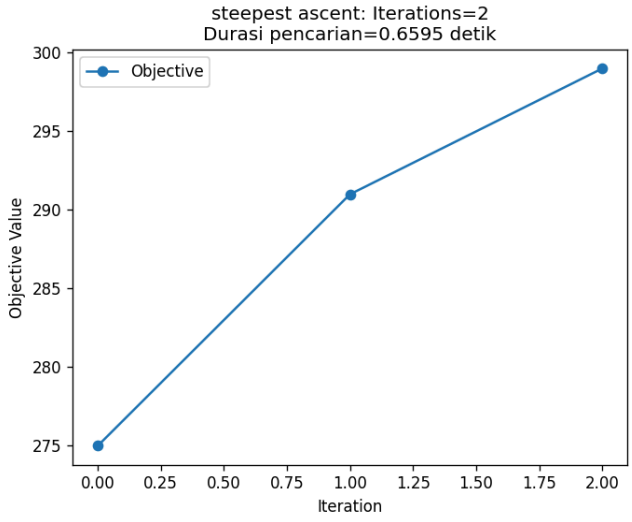
a. Steepest Ascent Hill-climbing

Eksperimen Ke-1	
State Awal	"Avanza": [5, 9, "V"], "Brio": null, "Fortuner": [3, 13, "H"], "Truk-Engkel": [0, 9, "V"], "Pajero": null, "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": [6, 15, "H"], "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
State Akhir	"Avanza": null, "Brio": [6, 15, "H"], "Fortuner": null, "Truk-Engkel": [0, 9, "V"], "Pajero": [3, 13, "H"], "Xpander": [5, 9, "V"], "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": null, "Elf": null,

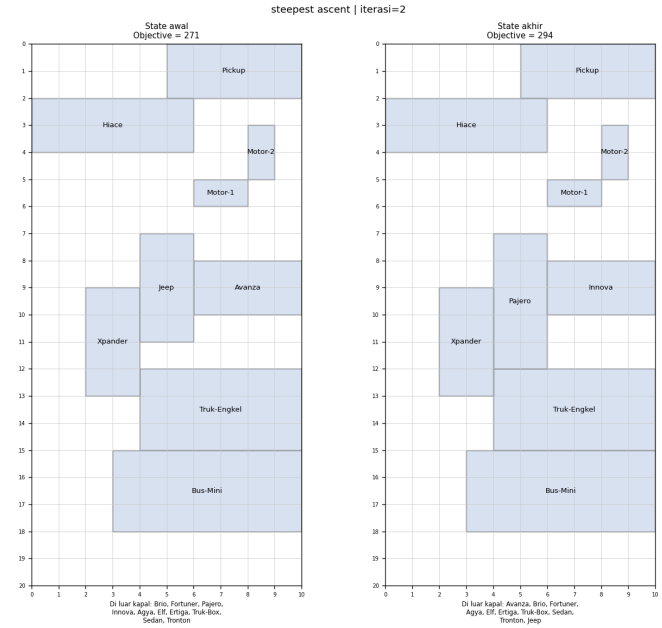
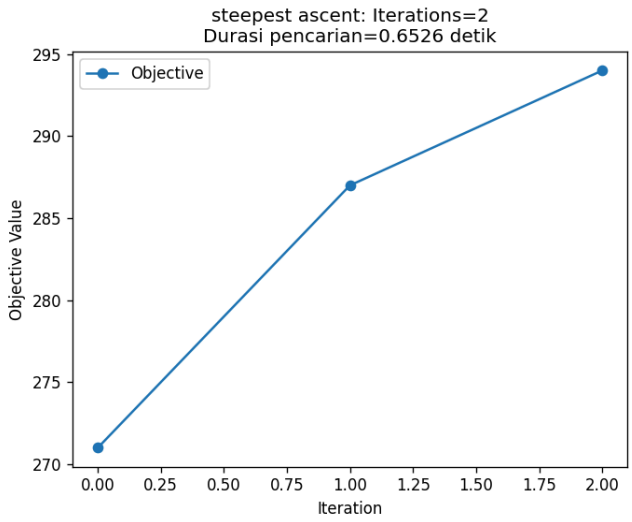
	"Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	300
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	2.2996 detik

Banyak iterasi hingga proses pencarian berhenti	3

Eksperimen Ke-2	
State Awal	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": null, "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": [7, 0, "V"], "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": [3, 9, "H"]
State Akhir	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [3, 9, "H"], "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": null, "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": [7, 0, "V"], "Truk-Box": null, "Sedan": [1, 13, "H"],

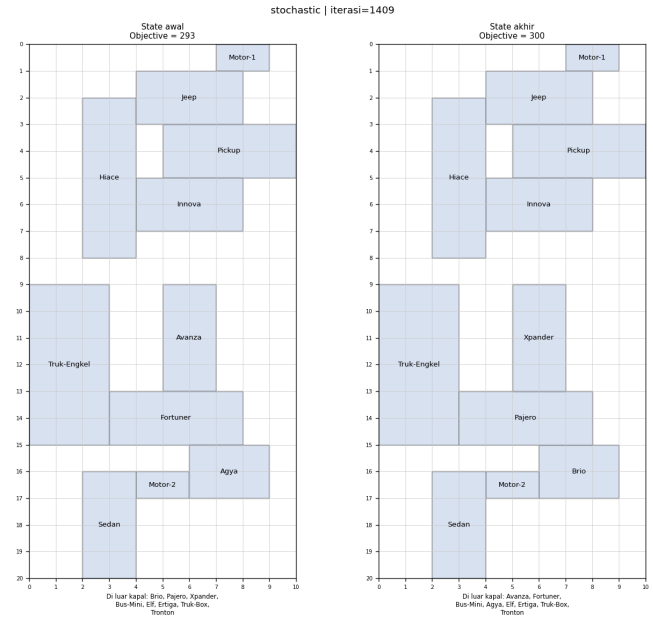
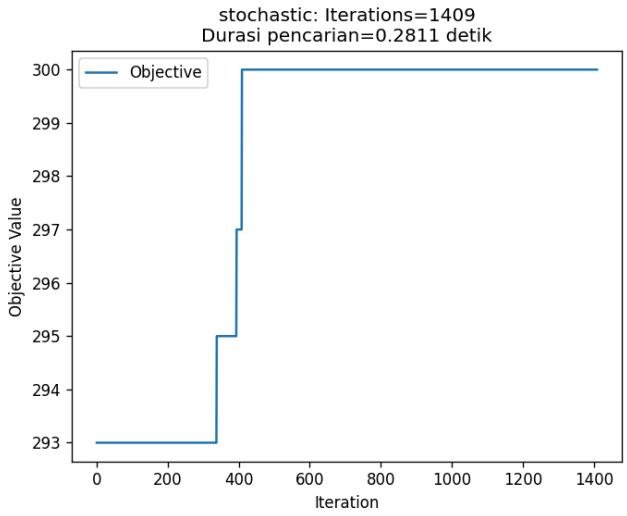
	"Tronton": [0, 1, "V"], "Jeep": null
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	299
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.6595 detik
Banyak iterasi hingga proses pencarian berhenti	2

Eksperimen Ke-3	
State Awal	"Avanza": [6, 8, "H"], "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": null, "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": null, "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [4, 7, "V"]
State Akhir	"Avanza": null, "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": [4, 7, "V"], "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": [6, 8, "H"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": null

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	294
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.6526 detik
Banyak iterasi hingga proses pencarian berhenti	2

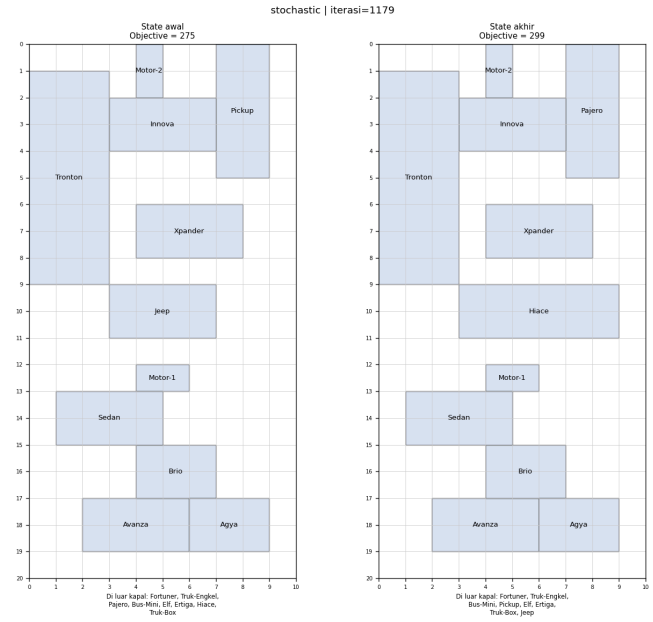
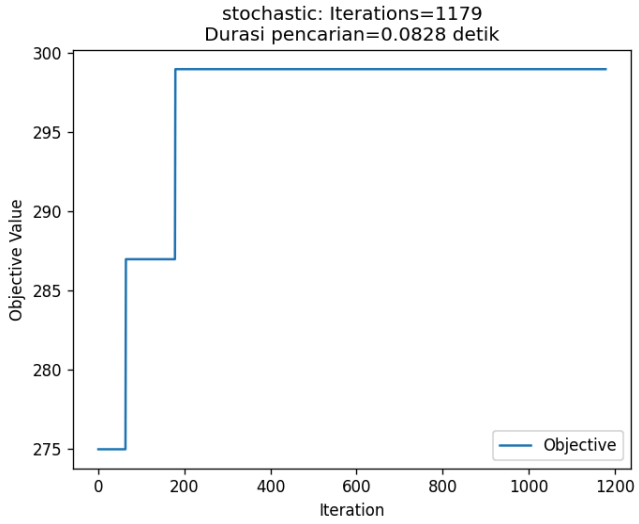
b. Stochastic Hill-climbing

Eksperimen Ke-1	
State Awal	"Avanza": [5, 9, "V"], "Brio": null, "Fortuner": [3, 13, "H"], "Truk-Engkel": [0, 9, "V"], "Pajero": null, "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": [6, 15, "H"], "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
State Akhir	"Avanza": null, "Brio": [6, 15, "H"], "Fortuner": null, "Truk-Engkel": [0, 9, "V"], "Pajero": [3, 13, "H"], "Xpander": [5, 9, "V"], "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]

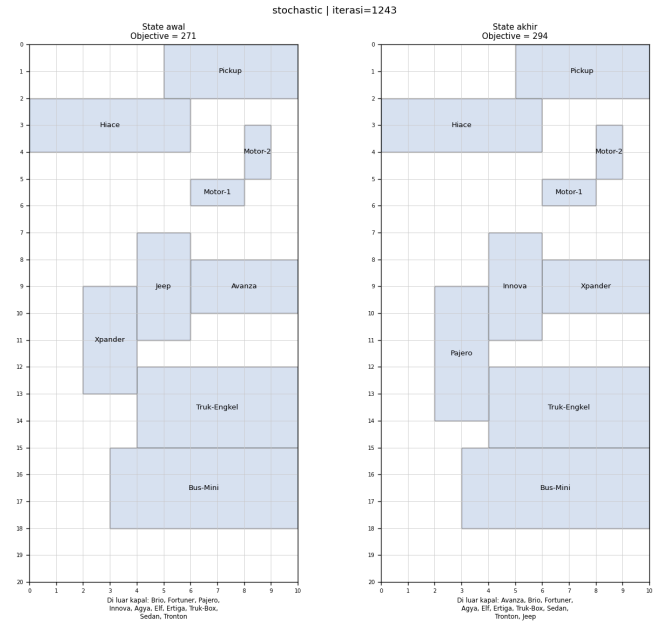
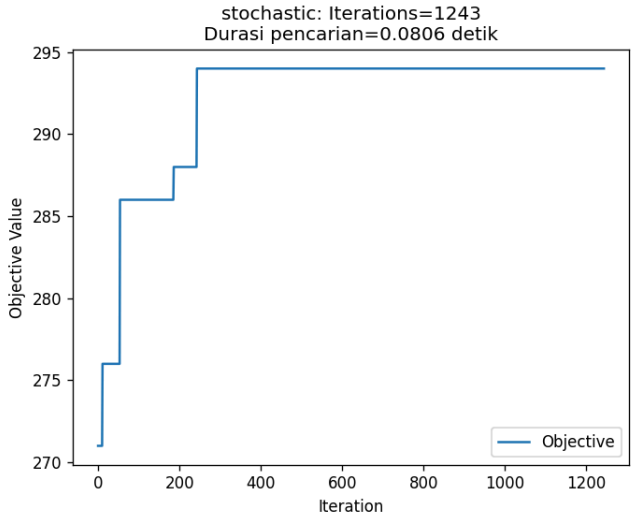
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	300
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.2811 detik
Banyak iterasi hingga proses pencarian berhenti	1.409

Eksperimen Ke-2

State Awal	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": null, "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": [7, 0, "V"], "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": [3, 9, "H"]
State Akhir	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [7, 0, "V"], "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": null, "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": [3, 9, "H"], "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": null

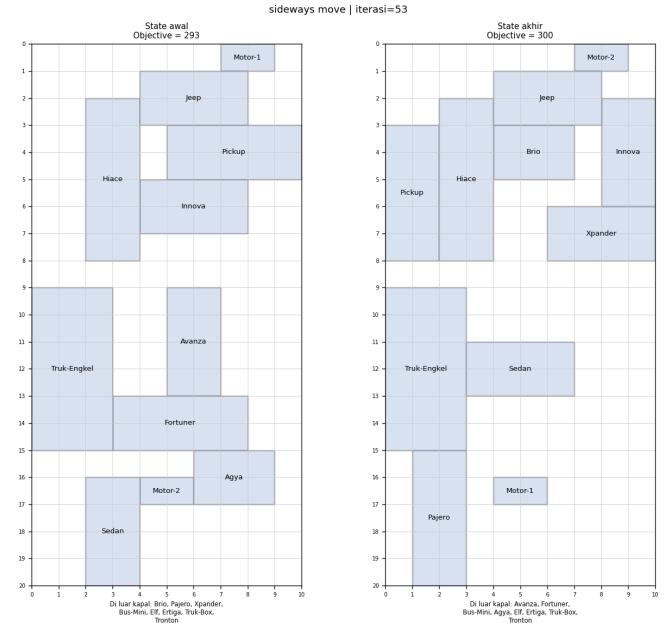
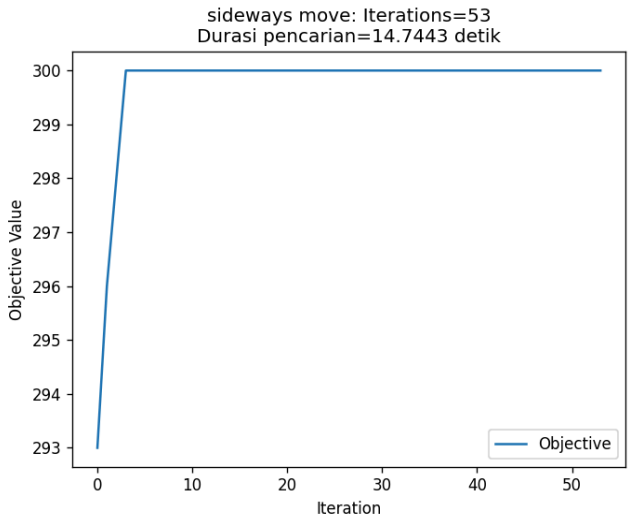
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	299
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.0828 detik
Banyak iterasi hingga proses pencarian berhenti	1.179

Eksperimen Ke-3	
State Awal	"Avanza": [6, 8, "H"], "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": null, "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": null, "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [4, 7, "V"]
State Akhir	"Avanza": null, "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": [2, 9, "V"], "Xpander": [6, 8, "H"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": [4, 7, "V"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": null

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	294
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.0806 detik
Banyak iterasi hingga proses pencarian berhenti	1.243

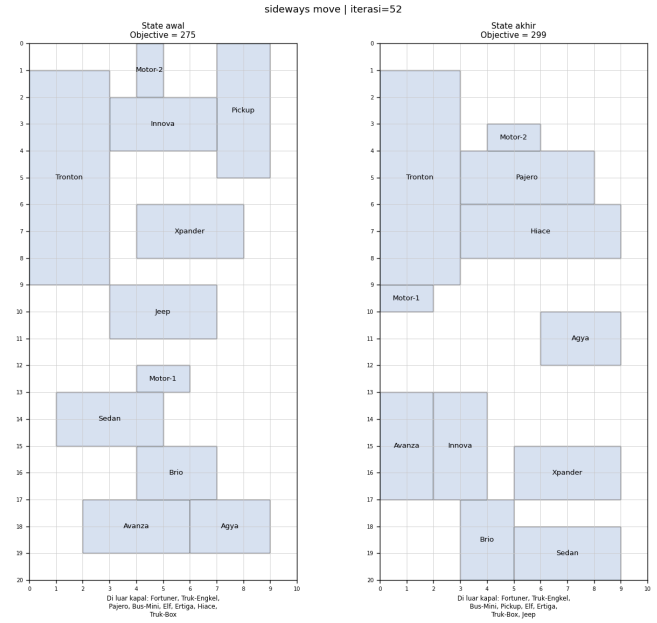
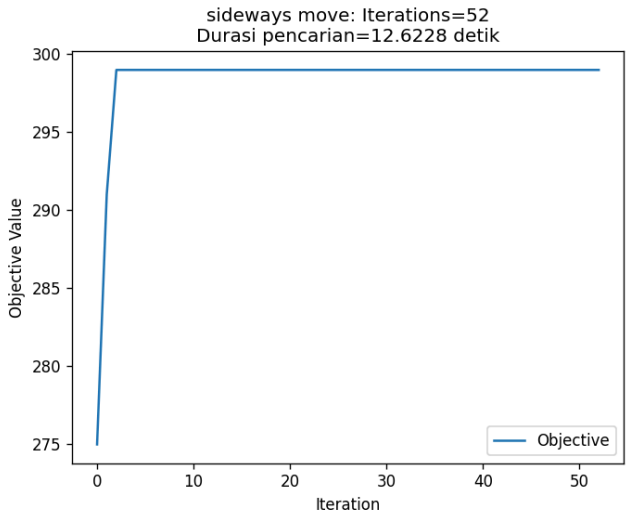
c. Hill-climbing with Sideways Move

Eksperimen Ke-1	
State Awal	"Avanza": [5, 9, "V"], "Brio": null, "Fortuner": [3, 13, "H"], "Truk-Engkel": [0, 9, "V"], "Pajero": null, "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": [6, 15, "H"], "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
State Akhir	"Avanza": null, "Brio": [4, 3, "H"], "Fortuner": null, "Truk-Engkel": [0, 9, "V"], "Pajero": [1, 15, "V"], "Xpander": [6, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 16, "H"], "Motor-2": [7, 0, "H"], "Pickup": [0, 3, "V"], "Innova": [8, 2, "V"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [3, 11, "H"], "Tronton": null, "Jeep": [4, 1, "H"]

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	300
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	14.7443 detik
Banyak iterasi hingga proses pencarian berhenti	53

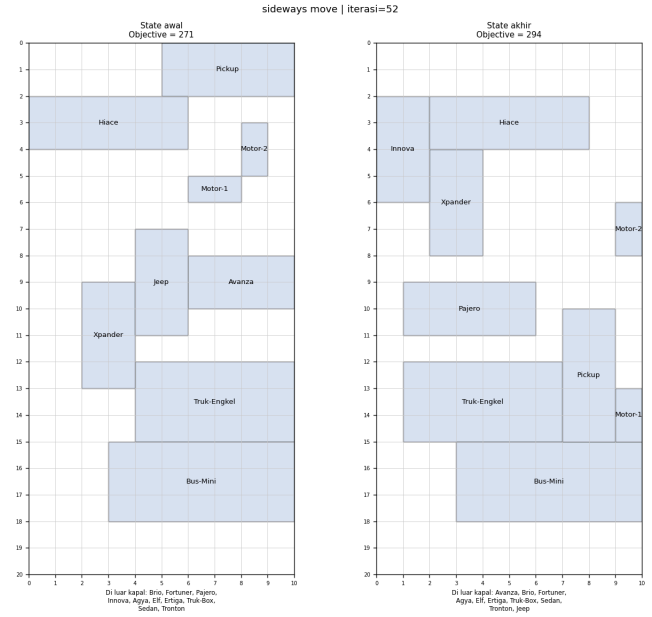
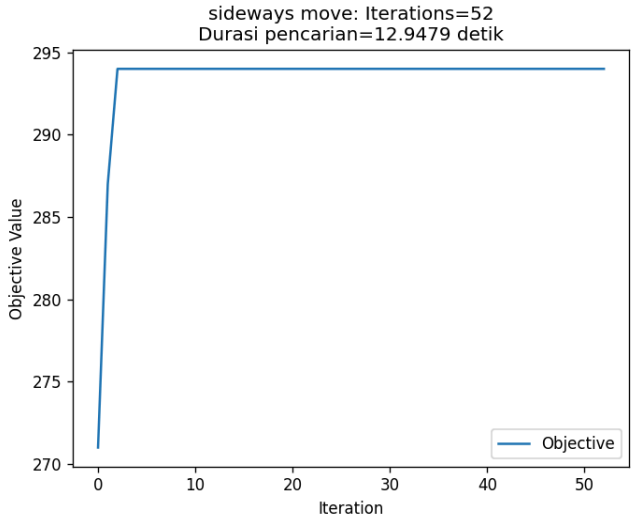
Eksperimen Ke-2

State Awal	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": null, "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": [7, 0, "V"], "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": [3, 9, "H"]
State Akhir	"Avanza": [0, 13, "V"], "Brio": [3, 17, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [3, 4, "H"], "Xpander": [5, 15, "H"], "Bus-Mini": null, "Motor-1": [0, 9, "H"], "Motor-2": [4, 3, "H"], "Pickup": null, "Innova": [2, 13, "V"], "Agya": [6, 10, "H"], "Elf": null, "Ertiga": null, "Hiace": [3, 6, "H"], "Truk-Box": null, "Sedan": [5, 18, "H"], "Tronton": [0, 1, "V"], "Jeep": null

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	299
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	12.6228 detik
Banyak iterasi hingga proses pencarian berhenti	52

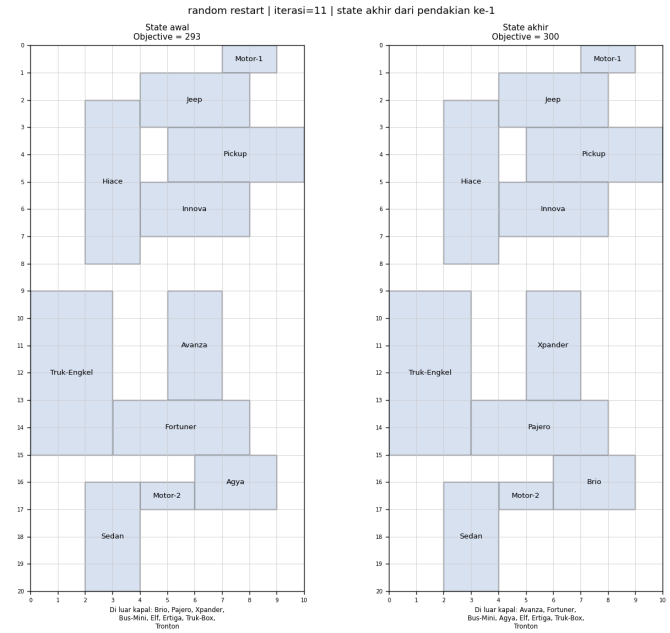
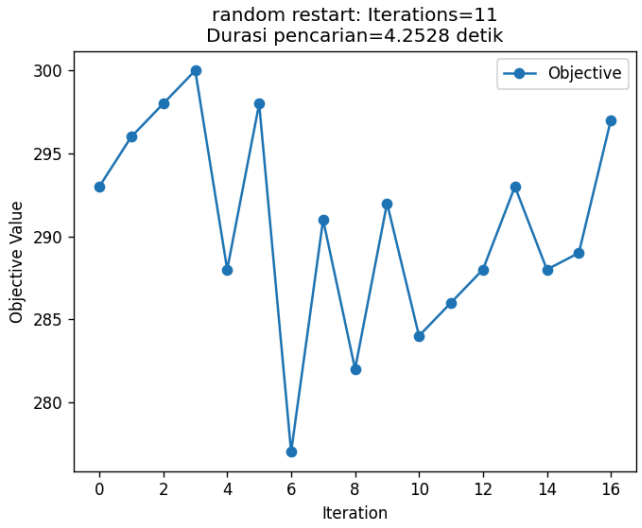
Eksperimen Ke-3

State Awal	"Avanza": [6, 8, "H"], "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": null, "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": null, "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [4, 7, "V"]
State Akhir	"Avanza": null, "Brio": null, "Fortuner": null, "Truk-Engkel": [1, 12, "H"], "Pajero": [1, 9, "H"], "Xpander": [2, 4, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [9, 13, "V"], "Motor-2": [9, 6, "V"], "Pickup": [7, 10, "V"], "Innova": [0, 2, "V"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [2, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": null

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	294
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	12.9479 detik
Banyak iterasi hingga proses pencarian berhenti	52

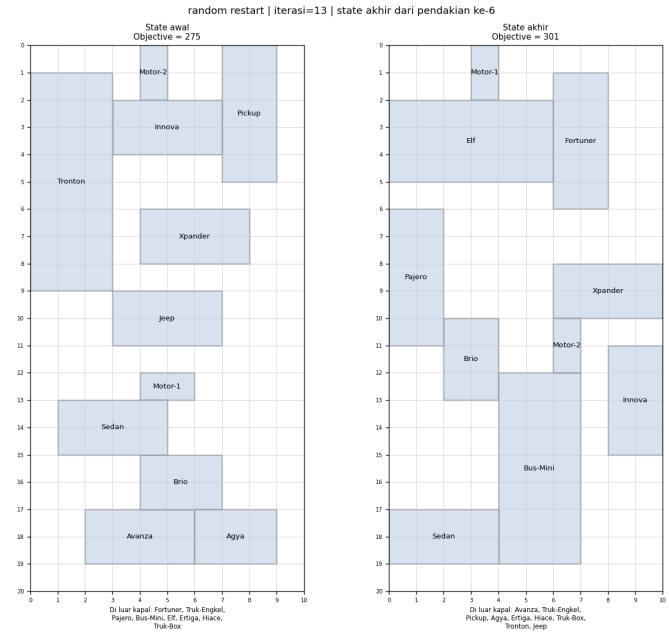
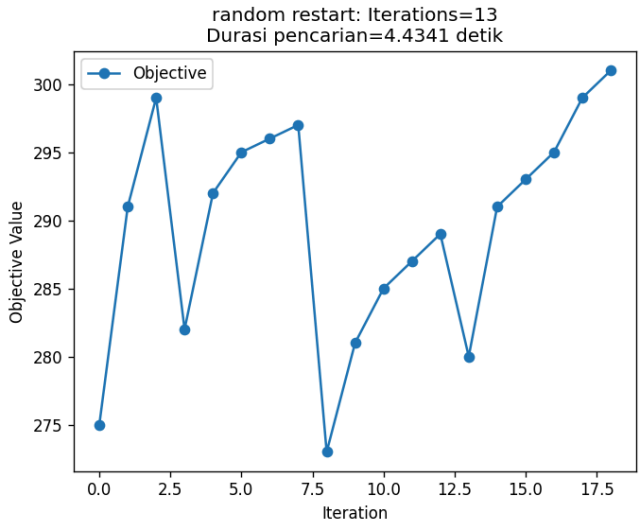
d. Random Restart Hill-climbing

Eksperimen Ke-1	
State Awal	"Avanza": [5, 9, "V"], "Brio": null, "Fortuner": [3, 13, "H"], "Truk-Engkel": [0, 9, "V"], "Pajero": null, "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": [6, 15, "H"], "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
State Akhir	"Avanza": null, "Brio": [6, 15, "H"], "Fortuner": null, "Truk-Engkel": [0, 9, "V"], "Pajero": [3, 13, "H"], "Xpander": [5, 9, "V"], "Bus-Mini": null, "Motor-1": [7, 0, "H"], "Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	300
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	4.2528 detik
Banyak restart	5

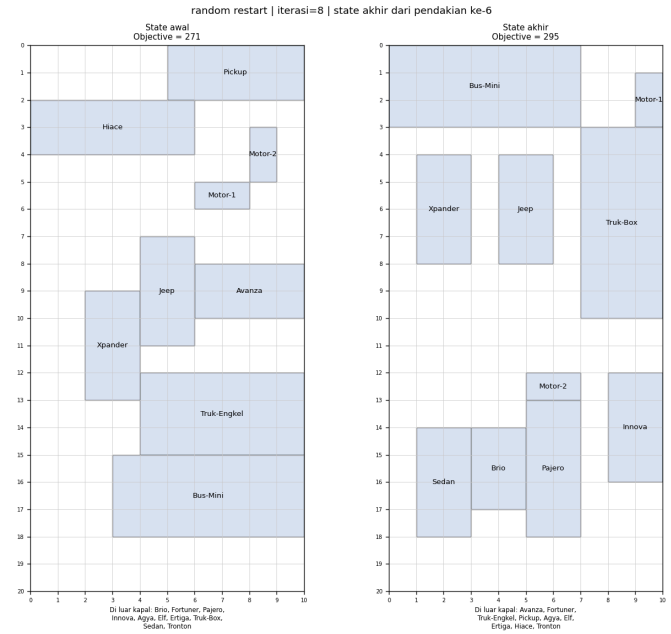
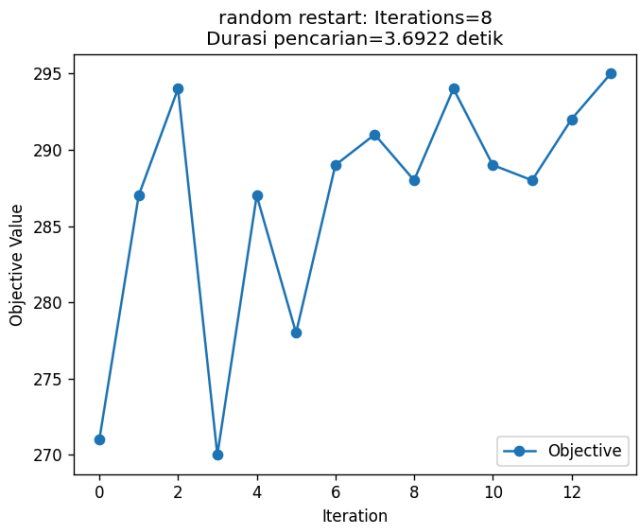
Eksperimen Ke-2

State Awal	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": null, "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": [7, 0, "V"], "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": [3, 9, "H"]
State Akhir	"Avanza": null, "Brio": [2, 10, "V"], "Fortuner": [6, 1, "V"], "Truk-Engkel": null, "Pajero": [0, 6, "V"], "Xpander": [6, 8, "H"], "Bus-Mini": [4, 12, "V"], "Motor-1": [3, 0, "V"], "Motor-2": [6, 10, "V"], "Pickup": null, "Innova": [8, 11, "V"], "Agya": null, "Elf": [0, 2, "H"], "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [0, 17, "H"], "Tronton": null, "Jeep": null

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	301
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	4.4341 detik
Banyak restart	5

Eksperimen Ke-3

State Awal	"Avanza": [6, 8, "H"], "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": null, "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": null, "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [4, 7, "V"]
State Akhir	"Avanza": null, "Brio": [3, 14, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [5, 13, "V"], "Xpander": [1, 4, "V"], "Bus-Mini": [0, 0, "H"], "Motor-1": [9, 1, "V"], "Motor-2": [5, 12, "H"], "Pickup": null, "Innova": [8, 12, "V"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": [7, 3, "V"], "Sedan": [1, 14, "V"], "Tronton": null, "Jeep": [4, 4, "V"]

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	295
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	3.6922 detik
Banyak restart	5

Ringkasan Hasil Hill-climbing

Varian	Eksperimen Ke-1	Eksperimen Ke-2	Eksperimen Ke-3	Rata-rata durasi

Nilai objektif awal	293	275	271	-
Steepest Ascent	300	299	294	1.2039 detik
Stochastic	300	299	294	0.1482 detik
Sideways Move	300	299	294	13.4383 detik
Random Restart	300	301	295	4.1264 detik

Analisis Hasil Hill-climbing

Initial state dibangkitkan secara acak dengan menempatkan kendaraan satu per satu pada posisi acak yang tidak bertumpuk selama total berat belum melebihi kapasitas. Akibatnya, seluruh initial state sudah valid dan total beratnya sudah mendekati kapasitas, yaitu 86 sampai 90 dari 90. Nilai objektif awal yang berkisar 271 sampai 293 pun sudah cukup tinggi, sehingga Hill-climbing hanya membutuhkan sedikit perpindahan untuk mencapai local optimum. Steepest Ascent hanya membutuhkan 2 sampai 3 iterasi dengan kenaikan nilai objektif sebesar 7 sampai 24.

Batas atas nilai objektif dihitung dengan mengabaikan posisi kendaraan dan hanya mempertimbangkan kapasitas berat, yaitu memilih kombinasi kendaraan dengan total ShippingFee terbesar yang total beratnya tidak melebihi 90. Batas atas yang diperoleh adalah 305. Nilai ini juga berhasil dicapai oleh Simulated Annealing pada Eksperimen Ke-1, sehingga 305 merupakan global optimum dari dataset ini. Hasil terbaik Hill-climbing adalah 301 (Random Restart, Eksperimen Ke-2) atau 98,7 persen dari global optimum, sedangkan hasil terendah adalah 294 (96,4 persen). Tidak ada varian Hill-climbing yang mencapai global optimum pada ketiga eksperimen.

Pada seluruh state akhir, total berat sudah mencapai 90 dari 90, sedangkan luas geladak yang terpakai hanya 91 sampai 94 dari 200 petak. Artinya, batasan yang paling menentukan pada dataset ini adalah kapasitas berat, bukan ruang geladak. Karena kapasitas berat sudah penuh, kenaikan nilai objektif hanya mungkin dengan mengganti kombinasi kendaraan, misalnya mengeluarkan beberapa kendaraan ringan untuk memasukkan kendaraan yang lebih mahal. Penggantian seperti ini membutuhkan lebih dari satu langkah, dan langkah pertamanya, yaitu mengeluarkan kendaraan, menurunkan nilai objektif. Hill-climbing tidak pernah memilih langkah yang menurunkan nilai, sehingga pencarian berhenti di local optimum. Hal inilah yang membedakan Hill-climbing dengan Simulated Annealing, yang masih dapat menerima langkah memburuk dengan peluang tertentu.

Pada ketiga eksperimen, Stochastic menghasilkan nilai akhir yang sama persis dengan Steepest Ascent, tetapi dengan durasi yang jauh lebih singkat karena setiap iterasi hanya mengevaluasi satu neighbor, sedangkan Steepest Ascent

mengevaluasi sekitar 3.000 neighbor per iterasi. Banyak iterasi Stochastic jauh lebih besar, yaitu 1.179 sampai 1.409, tetapi sekitar 1.000 iterasi terakhir merupakan percobaan gagal berturut-turut sebelum pencarian dihentikan oleh max_fail. Nilai max_fail = 1000 dipilih berdasarkan uji coba: dengan nilai 50 atau 100, pencarian berhenti sebelum mencapai local optimum sehingga nilai akhirnya lebih rendah, sedangkan mulai nilai 500 nilai akhir tidak berubah lagi.

Sideways Move menghasilkan nilai yang sama persis dengan Steepest Ascent, dengan durasi sekitar sepuluh kali lebih lama. Seluruh 50 langkah menyamping terpakai tanpa menghasilkan kenaikan nilai. Langkah menyamping pada persoalan ini umumnya berupa pemindahan kendaraan ke petak kosong lain yang tidak mengubah total berat. Karena batasan yang menentukan adalah berat dan ruang geladak masih banyak tersisa, langkah tersebut tidak membuka jalan menuju solusi yang lebih baik.

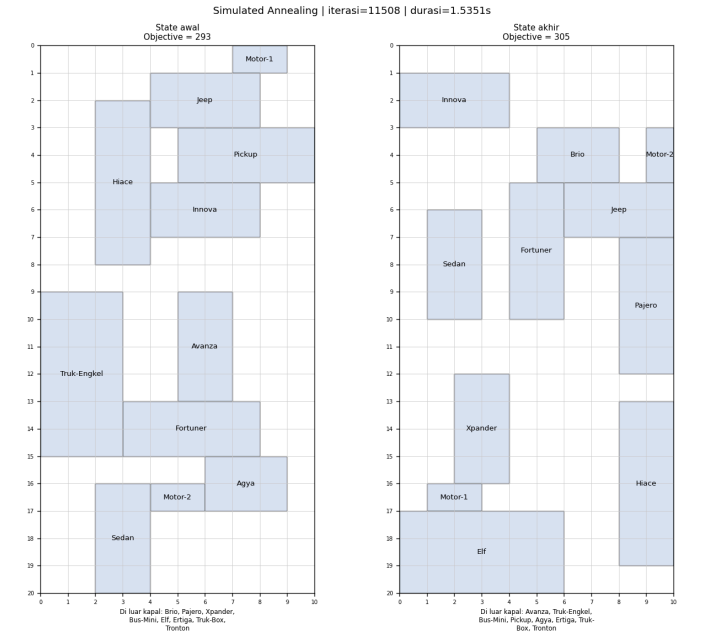
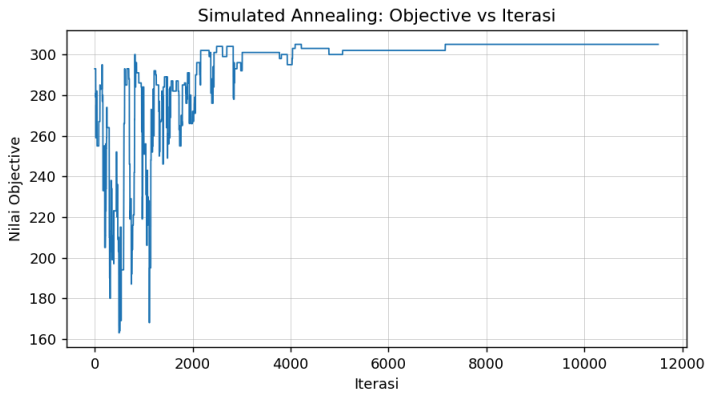
Random Restart merupakan satu-satunya varian yang menghasilkan nilai lebih tinggi dari Steepest Ascent, yaitu pada Eksperimen Ke-2 (301 dibanding 299) dan Eksperimen Ke-3 (295 dibanding 294). Peningkatan tersebut diperoleh dari pendakian ke-6 yang dimulai dari initial state berbeda, sehingga state akhir pada visualisasi bukan kelanjutan dari state awal yang ditampilkan. Pada Eksperimen Ke-1, pendakian pertama sudah memberikan hasil terbaik, yaitu 300. Peningkatan kualitas ini dibayar dengan durasi sekitar 3,5 kali lebih lama dari Steepest Ascent, karena Steepest Ascent dijalankan sebanyak 6 kali.

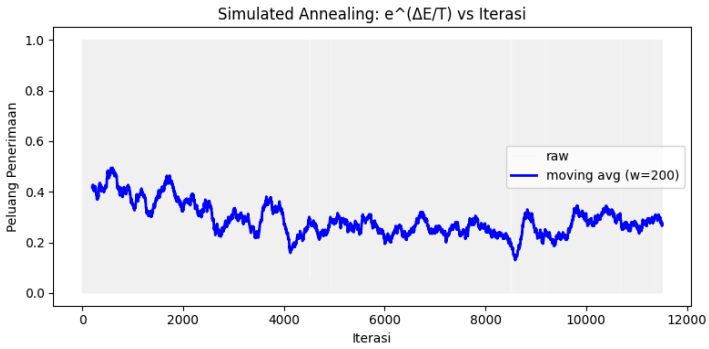
Untuk initial state yang sama, Steepest Ascent, Stochastic, dan Sideways Move menghasilkan nilai yang identik, sedangkan Random Restart lebih bervariasi karena menjelajahi beberapa initial state. Antar eksperimen, selisih nilai akhir setiap varian berkisar 6 sampai 7, sehingga hasil Hill-climbing cukup bergantung pada initial state. Durasi Eksperimen Ke-1 pada Steepest Ascent dan Stochastic tercatat lebih lama dari dua eksperimen lainnya, kemungkinan karena proses awal program saat fungsi pertama kali dijalankan.

2. Algoritma Simulated Annealing

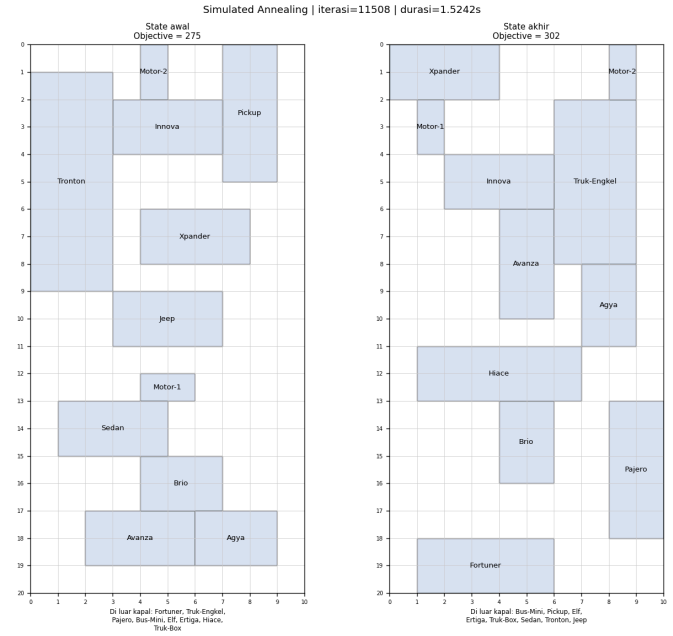
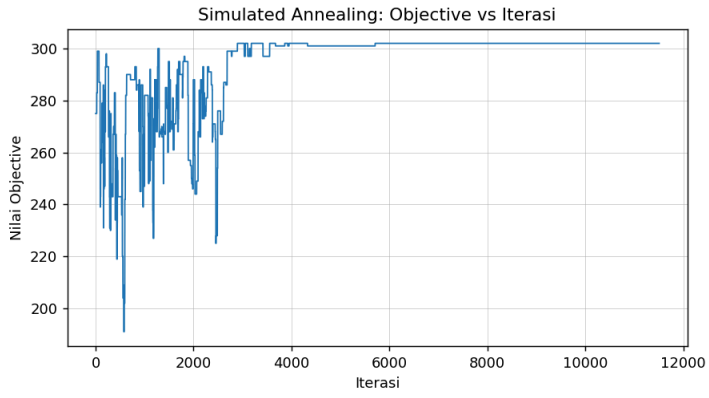
Eksperimen Ke-1	
State Awal	"Avanza": [5, 9, "V"], "Brio": null, "Fortuner": [3, 13, "H"], "Truk-Engkel": [0, 9, "V"], "Pajero": null, "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 0, "H"],

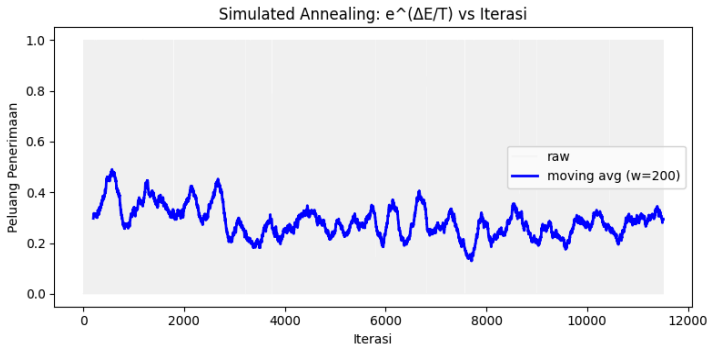
	"Motor-2": [4, 16, "H"], "Pickup": [5, 3, "H"], "Innova": [4, 5, "H"], "Agya": [6, 15, "H"], "Elf": null, "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [2, 16, "V"], "Tronton": null, "Jeep": [4, 1, "H"]
State Akhir	"Avanza": null, "Brio": [5, 3, "H"], "Fortuner": [4, 5, "V"], "Truk-Engkel": null, "Pajero": [8, 7, "V"], "Xpander": [2, 12, "V"], "Bus-Mini": null, "Motor-1": [1, 16, "H"], "Motor-2": [9, 3, "V"], "Pickup": null, "Innova": [0, 1, "H"], "Agya": null, "Elf": [0, 17, "H"], "Ertiga": null, "Hiace": [8, 13, "V"], "Truk-Box": null, "Sedan": [1, 6, "V"], "Tronton": null, "Jeep": [6, 5, "H"]

Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	305
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.6097 detik

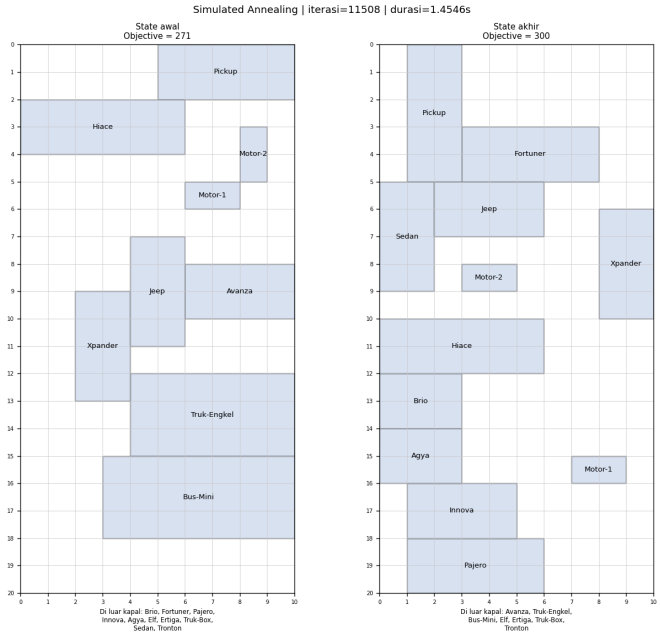
Plot peluang penerimaan $e^{dE/T}$ terhadap banyak iterasi	
Frekuensi stuck di local optima	40 kejadian

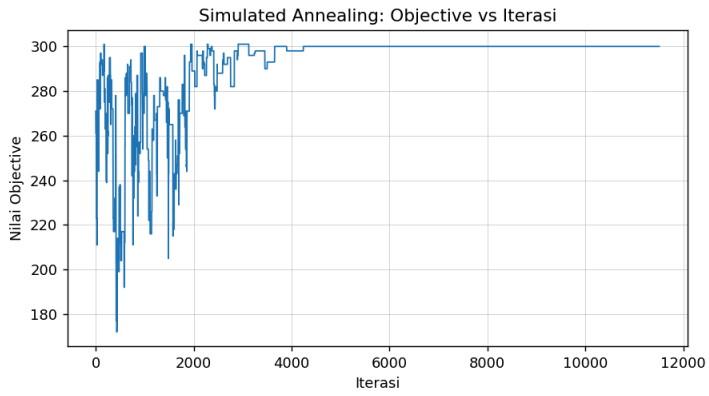
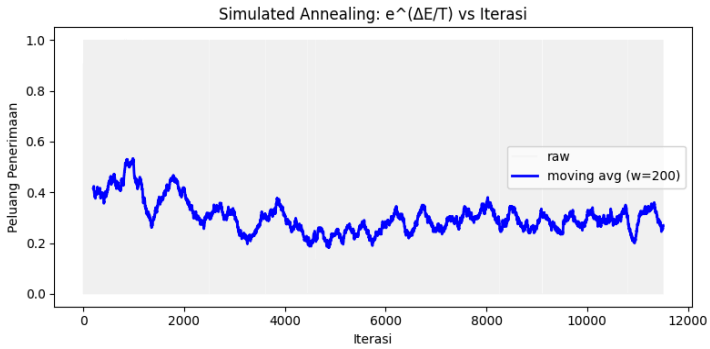
Eksperimen Ke-2	
State Awal	"Avanza": [2, 17, "H"], "Brio": [4, 15, "H"], "Fortuner": null, "Truk-Engkel": null, "Pajero": null, "Xpander": [4, 6, "H"], "Bus-Mini": null, "Motor-1": [4, 12, "H"], "Motor-2": [4, 0, "V"], "Pickup": [7, 0, "V"], "Innova": [3, 2, "H"], "Agya": [6, 17, "H"], "Elf": null, "Ertiga": null, "Hiace": null, "Truk-Box": null, "Sedan": [1, 13, "H"], "Tronton": [0, 1, "V"], "Jeep": [3, 9, "H"]
State Akhir	"Avanza": [4, 6, "V"], "Brio": [4, 13, "V"], "Fortuner": [1, 18, "H"], "Truk-Engkel": [6, 2, "V"], "Pajero": [8, 13, "V"], "Xpander": [0, 0, "H"], "Bus-Mini": null, "Motor-1": [1, 2, "V"],

	"Motor-2": [8, 0, "V"], "Pickup": null, "Innova": [2, 4, "H"], "Agya": [7, 8, "V"], "Elf": null, "Ertiga": null, "Hiace": [1, 11, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": null
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	302
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	

Durasi proses pencarian	0.6069 detik
Plot peluang penerimaan $e^{dE/T}$ terhadap banyak iterasi	
Frekuensi stuck di local optima	39 kejadian

Eksperimen Ke-3	
State Awal	"Avanza": [6, 8, "H"], "Brio": null, "Fortuner": null, "Truk-Engkel": [4, 12, "H"], "Pajero": null, "Xpander": [2, 9, "V"], "Bus-Mini": [3, 15, "H"], "Motor-1": [6, 5, "H"], "Motor-2": [8, 3, "V"], "Pickup": [5, 0, "H"], "Innova": null, "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [0, 2, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [4, 7, "V"]
State Akhir	"Avanza": null, "Brio": [0, 12, "H"], "Fortuner": [3, 3, "H"], "Truk-Engkel": null, "Pajero": [1, 18, "H"], "Xpander": [8, 6, "V"],

	"Bus-Mini": null, "Motor-1": [7, 15, "H"], "Motor-2": [3, 8, "H"], "Pickup": [1, 0, "V"], "Innova": [1, 16, "H"], "Agya": [0, 14, "H"], "Elf": null, "Ertiga": null, "Hiace": [0, 10, "H"], "Truk-Box": null, "Sedan": [0, 5, "V"], "Tronton": null, "Jeep": [2, 5, "H"]
Visualisasi State Awal dan Akhir	
Nilai objective function akhir yang dicapai	300

Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0,6204 detik
Plot peluang penerimaan $e^{dE/T}$ terhadap banyak iterasi	
Frekuensi stuck di local optima	40 kejadian

Analisis Hasil Simulated Annealing

Nilai T0 = 100 dan α = 0,999 ditentukan melalui hasil hasil parameter sweep yang dibahas rinci pada sub-bab Analisis Pengaruh Alpha dan Analisis Pengaruh T0 setelah ini. Secara ringkas, T0 = 100 dipilih karena memberikan peluang penerimaan awal yang cukup lebar sehingga SA punya ruang eksploratif memadai pada tahap awal pencarian, dan secara empiris merupakan satu-satunya konfigurasi yang berhasil menyentuh global optimum 305 di antara lima nilai T0 yang diuji (5, 10, 20, 50, 100). Sementara itu, α = 0,999 adalah titik keseimbangan terbaik antara kualitas solusi dan biaya komputasi dibandingkan nilai yang lebih kecil (eksplorasi terhenti terlalu dini pada α = 0,99 dengan hanya 1.146 iterasi) maupun lebih besar (biaya komputasi melonjak hampir sepuluh kali lipat pada α = 0,9999 tanpa peningkatan rata-rata yang berarti).

Parameter lain mengikuti konvensi standar implementasi SA. T_min = 10^{-3} menjadi ambang suhu rendah agar algoritma berhenti ketika SA praktis

sudah bekerja mirip hill-climbing. Nilai max_iter = 30.000 berperan sebagai pengaman jika suhu belum turun di bawah T_min dalam jumlah iterasi yang wajar, pada konfigurasi ini praktis tidak terpakai karena pencarian berhenti alami pada 11.508 iterasi akibat tercapainya T_min. Terakhir, stuck_window = 200 dipakai untuk mendeteksi stagnasi: SA dianggap stuck jika 200 iterasi berturut-turut tidak ada perbaikan, dan jumlah kejadiannya dilaporkan sebagai indikator seberapa sering pencarian terjebak pada local optima.

Ketiga eksperimen memakai konfigurasi parameter yang sama persis dan hanya nilai seed yang diubah. Karena itu, perbedaan hasil di antara ketiganya murni berasal dari sifat acak Simulated Annealing, yaitu perbedaan urutan tetangga yang dibangkitkan oleh pembangkit bilangan acak.

Hal yang paling menarik terlihat pada Eksperimen Ke-1 dengan seed 0 yang berhasil mencapai global optimum teoretis sebesar 305. Artinya, SA dengan konfigurasi *default* sudah bisa menemukan solusi terbaik pada seed yang tepat, tanpa perlu mengubah parameter atau menambah mekanisme seperti multi-restart. Dua eksperimen lainnya menghasilkan nilai 302 dan 300, yang masih sangat dekat dengan optimum (masing-masing 99 persen dan 98,4 persen). Selisih hasil yang hanya 5 unit dari global optimum menunjukkan bahwa SA cukup konsisten pada persoalan ini, meskipun seed diganti. Konsistensi ini dibahas lebih dalam pada sub-bab Analisis Konsistensi dengan sepuluh seed di laporan utama.

Dari sisi efisiensi, waktu eksekusi ketiga eksperimen hampir sama, yaitu sekitar 0,60 sampai 0,61 detik, dengan jumlah iterasi yang sama persis, yakni 11.508. Hal ini wajar, sebab jumlah iterasi SA ditentukan oleh T0, α, dan T_min melalui rumus $\frac{ln(T_min/T0)}{ln(\alpha)}$, sehingga seed tidak memengaruhi banyaknya langkah. Frekuensi stuck juga seragam, yaitu 39 sampai 40 kali pada ketiga eksperimen. Ini menandakan SA terjebak pada local optima dengan intensitas yang serupa. Dengan kata lain, seed tidak banyak mengubah dinamika pencarian, tetapi hanya menentukan di titik mana pencarian berakhir. Secara keseluruhan, SA dengan konfigurasi default menghasilkan solusi yang valid dan konsisten mendekati optimum teoretis.

Analisis Konsistensi dengan Sepuluh Seed

Salah satu karakteristik penting algoritma stokastik seperti Simulated Annealing adalah variabilitas hasil antarproses pencarian. Untuk menguji konsistensi SA secara lebih luas, dilakukan sepuluh percobaan dengan seed berbeda (0 sampai 9) pada dataset sample_medium menggunakan konfigurasi parameter yang sama dengan Eksperimen Ke-1 sampai Ke-3.

Seed	Nilai Awal	Nilai Akhir	Valid	Iterasi	Durasi
0	293	305	Ya	11.508	0,60 detik
1	275	302	Ya	11.508	0,60 detik
2	271	300	Ya	11.508	0,61 detik
3	290	300	Ya	11.508	0,61 detik
4	298	298	Ya	11.508	0,62 detik
5	285	295	Ya	11.508	0,59 detik
6	286	299	Ya	11.508	0,61 detik
7	268	290	Ya	11.508	0,61 detik
8	284	297	Ya	11.508	0,60 detik
9	275	301	Ya	11.508	0,61 detik

Statistik ringkas dari kesepuluh percobaan di atas disusun pada tabel berikut.

Statistik	Nilai
Minimum	290
Maksimum	305 (global optimum tercapai pada seed 0)
Rata-rata	298,7
Simpangan baku	3,90
Validitas	10/10 run

Tiga pengamatan utama dapat diambil dari data di atas. Pertama, rata-rata nilai akhir sebesar 298,7 mencapai 97,9 persen dari optimum teoretis, dan seed 0 berhasil menyentuh global optimum 305. Hal ini membuktikan bahwa SA dengan konfigurasi default tidak hanya menghasilkan solusi mendekati optimum, tetapi juga sanggup mencapainya tanpa modifikasi parameter.

Kedua, simpangan baku sebesar 3,90 poin (sekitar 1,3 persen dari rata-rata) menunjukkan karakter pencarian yang cukup konsisten antar seed. Rentang hasil 15 poin (290 sampai 305) tergolong sempit untuk persoalan diskret dengan ruang pencarian besar.

Ketiga, nilai awal yang bervariasi antara 268 (seed 7) sampai 298 (seed 4) ternyata tidak berkorelasi langsung dengan nilai akhir. Seed 4 yang memulai dari nilai awal tertinggi justru hanya mencapai 298, sedangkan seed 0 yang

memulai dari nilai menengah 293 malah menyentuh global optimum. Hal ini menegaskan bahwa karakter pencarian SA lebih bergantung pada jalur eksplorasi sepanjang iterasi, bukan pada kualitas state awal.

Dari sudut pandang efisiensi, durasi tiap percobaan hanya berkisar 0,59 sampai 0,62 detik. Keseragaman ini wajar karena jumlah iterasi SA ditentukan secara deterministik (11.508 iterasi) dan beban komputasi per iterasi relatif serupa.

Analisis Pengaruh Alpha (Cooling Rate)

Koefisien pendinginan alpha menentukan seberapa lambat suhu T menurun di setiap iterasi. Untuk mengukur dampaknya, dilakukan parameter sweep terhadap empat nilai alpha dengan tiga seed per nilai, menggunakan dataset sample_medium.

alpha	Nilai Akhir (seed 0, 1, 2)	Maksimum	Rata-rata	Iterasi rata-rata	Durasi rata-rata
0,99	298, 294, 297	298	296,3	1.146	0,06 detik
0,995	304, 300, 297	304	300,3	2.297	0,12 detik
0,999	305, 302, 300	305	302,3	11.508	0,60 detik
0,9999	305, 301, 299	305	301,7	115.124	5,98 detik

Peningkatan alpha dari 0,99 ke 0,995 memberikan lompatan kualitas yang signifikan, rata-rata naik dari 296,3 ke 300,3. Pada alpha serendah 0,99, SA hanya menjalankan 1.146 iterasi sebelum suhu turun di bawah T_min, sehingga eksplorasi terhenti terlalu dini dan algoritma tidak sempat menemukan solusi optimal.

Dari alpha 0,999 ke 0,9999, perbedaan rata-rata memang kecil (dari 302,3 menjadi 301,7), namun kedua konfigurasi berhasil menyentuh global optimum 305 pada seed 0. Perbedaan utama justru terletak pada biaya komputasi: alpha 0,9999 memerlukan hampir sepuluh kali lipat durasi (5,98 detik berbanding 0,60 detik) dengan hasil rata-rata yang sebenarnya tidak lebih baik secara statistik.

Temuan utama dari sweep ini adalah bahwa alpha 0,999 merupakan titik keseimbangan terbaik antara kualitas solusi dan biaya komputasi, sehingga dipilih sebagai nilai default pada implementasi laporan ini.

Analisis Pengaruh T0 (Suhu Awal)

Selain alpha, suhu awal T0 juga sangat memengaruhi perilaku SA, terutama pada tahap awal pencarian. Pemilihan nilai T0 pada implementasi

akhir dilakukan berdasarkan analisis teoretis dan empiris berikut. Secara teoretis, peluang penerimaan tetangga yang lebih buruk pada iterasi pertama dapat dihitung melalui rumus $e^{dE/T0}$. Tabel di bawah memperlihatkan nilai peluang ini untuk beberapa selisih kualitas pada T0 = 100.

Selisih dE	**Peluang penerimaan $e^{dE/T0}$**
−1	0,951 (hampir pasti diterima)
−5	0,779
−10	0,607
−20	0,368
−50	0,082

Dari tabel tersebut terlihat bahwa pada T0 = 100, peluang menerima tetangga yang jauh lebih buruk (selisih −50) hanya 0,007, kecil tetapi tidak nol. Peluang ini terus mengecil seiring menurunnya suhu, sehingga pada akhir pencarian SA berperilaku semakin mirip hill-climbing. Untuk memvalidasi pilihan ini secara empiris, dilakukan sweep terhadap lima nilai T0 dengan tiga seed per nilai.

T0	**Nilai Akhir (Seed 0, 1, 2)**	**Maksimum**	**Rata-rata**	**Durasi rata-rata**
5	301, 299, 294	301	298,0	0,46 detik
10	300, 293, 294	300	295,7	0,48 detik
20	297, 299, 304	304	300,0	0,52 detik
50	302, 301, 292	302	298,3	0,58 detik
100	305, 302, 300	305	302,3	0,61 detik

Hasil eksperimen di atas memberikan dua kesimpulan penting. Pertama, T0 = 100 memberikan hasil terbaik baik dari sisi maksimum maupun rata-rata, dan merupakan satu-satunya konfigurasi yang berhasil menyentuh global optimum. Fase eksplorasi yang lebih lama ternyata menguntungkan untuk persoalan ini. Kedua, T0 pada rentang kecil (5 sampai 20) menunjukkan hasil yang tidak bersifat monoton melainkan fluktuatif. Hal ini menandakan bahwa SA dengan suhu awal rendah sangat rentan terhadap jebakan local optima karena peluang menerima tetangga buruk sudah terlalu kecil sejak awal pencarian.

Berdasarkan eksperimen parameter sweep ini, T0 = 100 dipilih sebagai nilai default pada implementasi akhir laporan. Hasil ini menegaskan bahwa pemilihan T0 memiliki landasan empiris, bukan sekadar tebakan.

Analisis Skalabilitas

Untuk mengukur bagaimana kinerja SA bergantung pada ukuran persoalan, dilakukan perbandingan antara dataset sample_small (5 kendaraan, grid 4×5) dan sample_medium (19 kendaraan, grid 10×20), masing-masing dengan tiga seed.

Dataset	Seed	Nilai Awal	Nilai Akhir	Valid	Durasi
sample_small.json	0	47	52	Ya	0,18 detik
sample_small.json	1	37	52	Ya	0,17 detik
sample_small.json	2	52	52	Ya	0,18 detik
sample_medium.json	0	293	305	Ya	0,60 detik
sample_medium.json	1	275	302	Ya	0,60 detik
sample_medium.json	2	271	300	Ya	0,61 detik

Untuk kemudahan interpretasi, perbandingan dengan optimum teoretis diringkas pada tabel berikut.

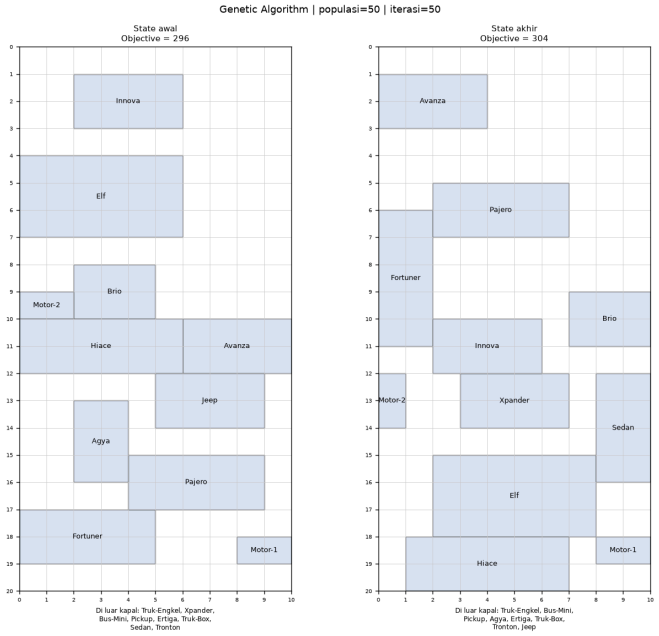
Dataset	Optimum Teoretis	SA Maksimum	Persen Pencapaian
sample_small.json	55 (angkut kendaraan B dan E)	52	94,5 persen
sample_medium.json	305	305	100 persen

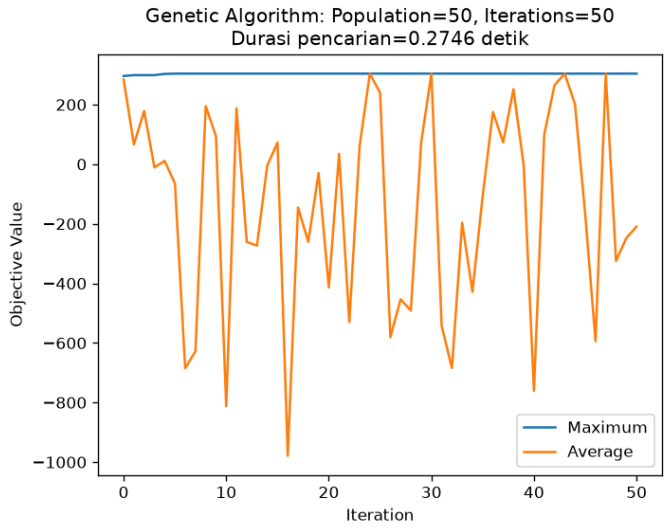
Terdapat satu temuan menarik dari sisi kualitas: SA justru mencapai persentase optimum yang lebih tinggi pada dataset medium (100 persen) dibandingkan small (94,5 persen). Penyebabnya terletak pada struktur persoalan: pada dataset small, kombinasi optimal mengharuskan hanya kendaraan B dan E yang diangkut agar grid 4×5 tidak melebihi kapasitas, sementara SA kerap terjebak pada kombinasi suboptimal karena ruang solusi yang sempit tidak banyak memberikan jalan keluar. Pada dataset medium, ruang pencarian jauh lebih luas, namun 19 kendaraan dan 11.508 iterasi memberi SA cukup banyak sudut untuk dieksplorasi. Secara praktis, hal ini membuat peluang mencapai global optimum justru lebih tinggi di persoalan yang lebih besar.

Dari sisi waktu komputasi, dataset medium (sekitar 0,60 detik) jauh lebih mahal dibandingkan dataset small (sekitar 0,18 detik) karena random_neighbor dan objective.value bekerja pada state dengan 19 slot kendaraan dan grid yang hampir tiga kali lipat lebih besar. Kesimpulan dari analisis skalabilitas ini adalah bahwa SA bersifat relatif skalabel untuk persoalan ini, dengan waktu komputasi yang bertambah proporsional terhadap ukuran persoalan tetapi tanpa ledakan eksponensial.

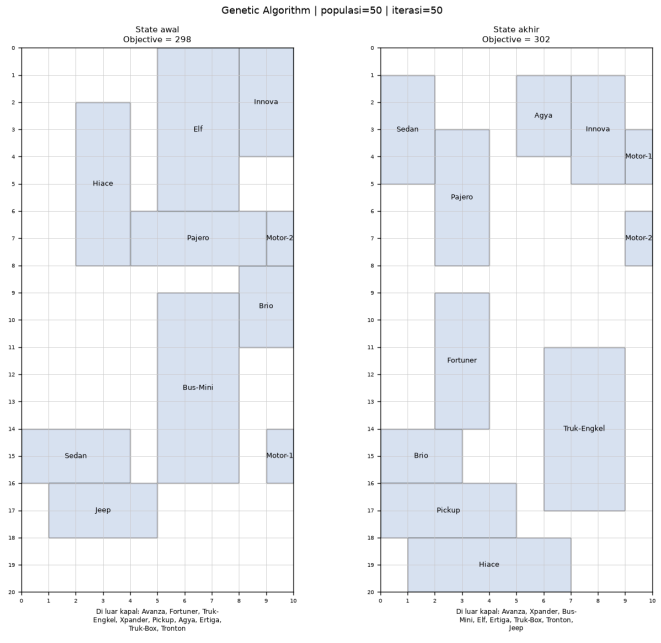
3. Algoritma Genetic
   a. Populasi sebagai kontrol

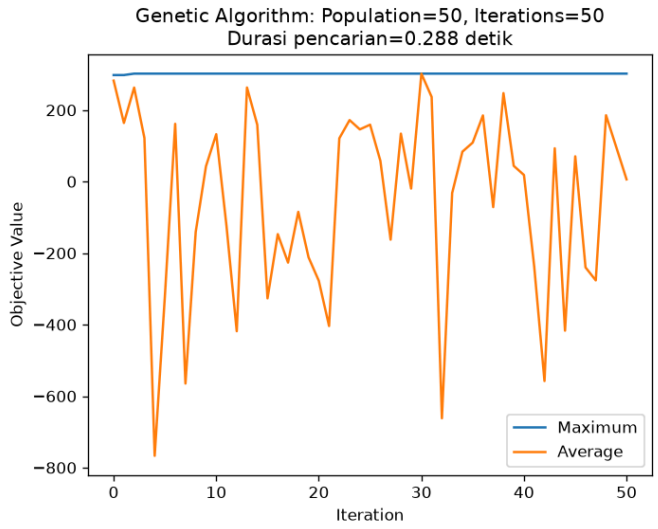
Eksperimen Ke-1	
Jumlah populasi	50
Banyak iterasi	50
State Awal	"Avanza": [6, 10, "H"], "Brio": [2, 8, "H"], "Fortuner": [0, 17, "H"], "Truk-Engkel": null, "Pajero": [4, 15, "H"], "Xpander": null, "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 9, "H"], "Pickup": null, "Innova": [2, 1, "H"], "Agya": [2, 13, "V"], "Elf": [0, 4, "H"], "Ertiga": null, "Hiace": [0, 10, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [5, 12, "H"]
State Akhir	"Avanza": [0, 1, "H"], "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": null, "Pajero": [2, 5, "H"], "Xpander": [3, 12, "H"],

	"Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 12, "V"], "Pickup": null, "Innova": [2, 10, "H"], "Agya": null, "Elf": [2, 15, "H"], "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

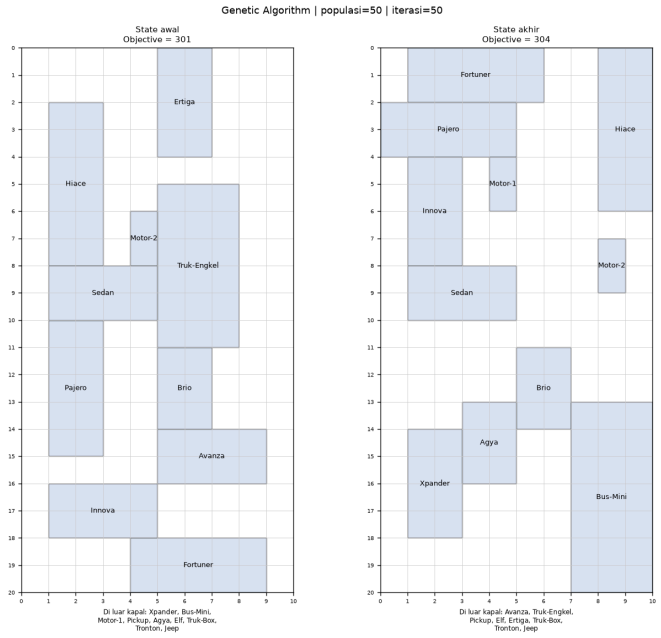
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.2746 detik

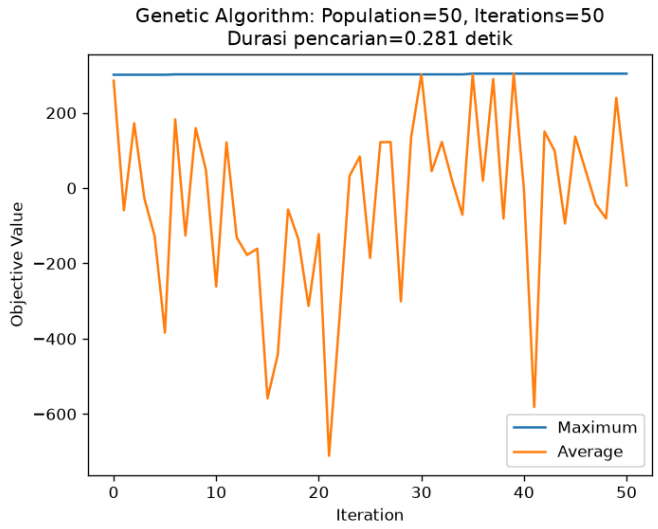
Eksperimen Ke-2	
Jumlah populasi	50
Banyak iterasi	50
State Awal	"Avanza": null, "Brio": [8, 8, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [4, 6, "H"], "Xpander": null, "Bus-Mini": [5, 9, "V"], "Motor-1": [9, 14, "V"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [8, 0, "V"], "Agya": null, "Elf": [5, 0, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [1, 16, "H"]

State Akhir	"Avanza": null, "Brio": [0, 14, "H"], "Fortuner": [2, 9, "V"], "Truk-Engkel": [6, 11, "V"], "Pajero": [2, 3, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": [9, 3, "V"], "Motor-2": [9, 6, "V"], "Pickup": [0, 16, "H"], "Innova": [7, 1, "V"], "Agya": [5, 1, "V"], "Elf": null, "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [0, 1, "V"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	302

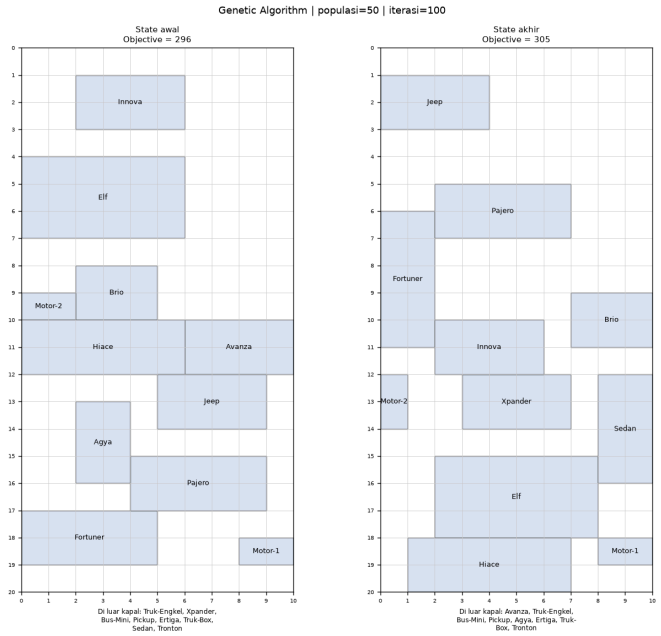
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.288 detik

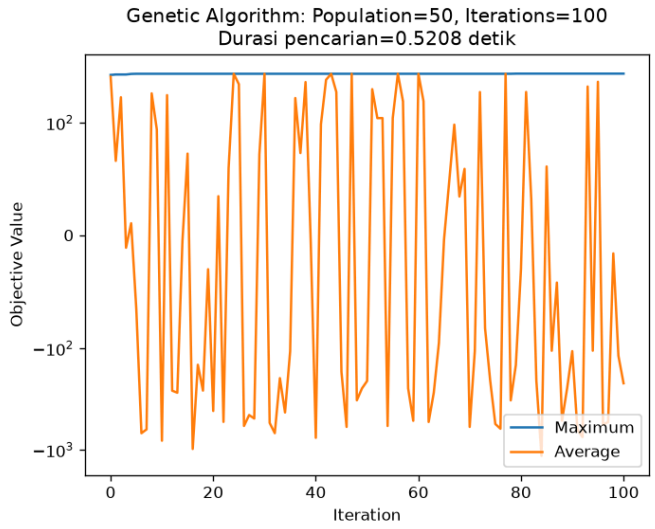
Eksperimen Ke-3	
Jumlah populasi	50
Banyak iterasi	50
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [5, 11, "V"], "Fortuner": [1, 0, "H"], "Truk-Engkel": null, "Pajero": [0, 2, "H"], "Xpander": [1, 14, "V"], "Bus-Mini": [7, 13, "V"], "Motor-1": [4, 4, "V"], "Motor-2": [8, 7, "V"], "Pickup": null, "Innova": [1, 4, "V"], "Agya": [3, 13, "V"], "Elf": null, "Ertiga": null, "Hiace": [8, 0, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

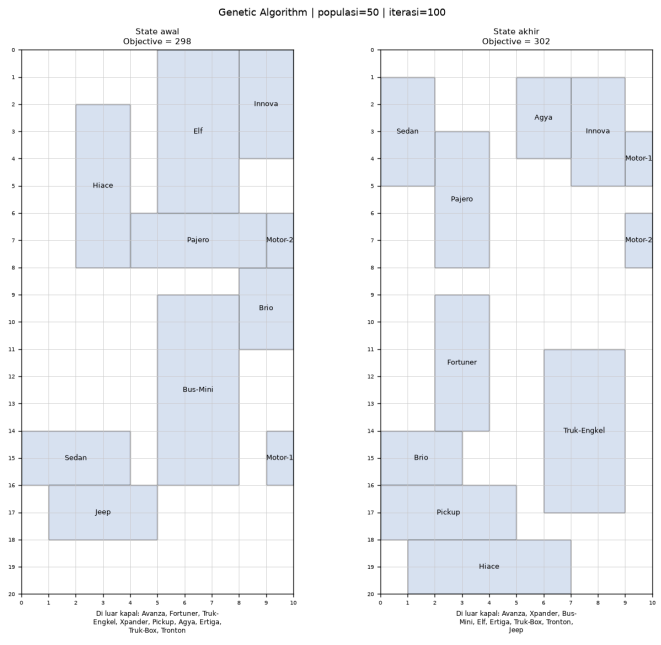
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.281 detik

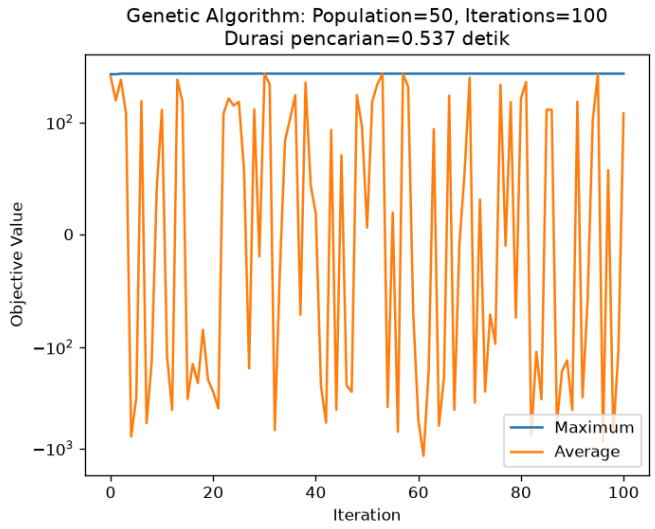
Eksperimen Ke-4	
Jumlah populasi	50
Banyak iterasi	100
State Awal	"Avanza": [6, 10, "H"], "Brio": [2, 8, "H"], "Fortuner": [0, 17, "H"], "Truk-Engkel": null, "Pajero": [4, 15, "H"], "Xpander": null, "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 9, "H"], "Pickup": null, "Innova": [2, 1, "H"], "Agya": [2, 13, "V"], "Elf": [0, 4, "H"], "Ertiga": null, "Hiace": [0, 10, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [5, 12, "H"]

State Akhir	"Avanza": null, "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": null, "Pajero": [2, 5, "H"], "Xpander": [3, 12, "H"], "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 12, "V"], "Pickup": null, "Innova": [2, 10, "H"], "Agya": null, "Elf": [2, 15, "H"], "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": [0, 1, "H"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

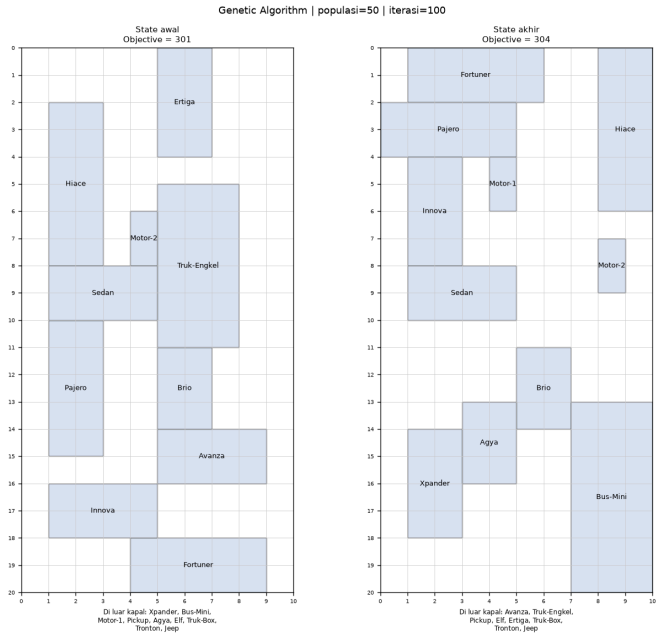
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.5208 detik

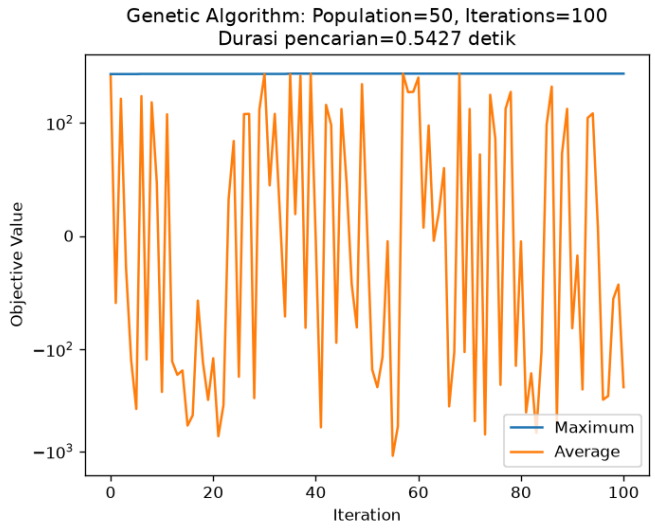
Eksperimen Ke-5	
Jumlah populasi	50
Banyak iterasi	100
State Awal	"Avanza": null, "Brio": [8, 8, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [4, 6, "H"], "Xpander": null, "Bus-Mini": [5, 9, "V"], "Motor-1": [9, 14, "V"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [8, 0, "V"], "Agya": null, "Elf": [5, 0, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [1, 16, "H"]

State Akhir	"Avanza": null, "Brio": [0, 14, "H"], "Fortuner": [2, 9, "V"], "Truk-Engkel": [6, 11, "V"], "Pajero": [2, 3, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": [9, 3, "V"], "Motor-2": [9, 6, "V"], "Pickup": [0, 16, "H"], "Innova": [7, 1, "V"], "Agya": [5, 1, "V"], "Elf": null, "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [0, 1, "V"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	302

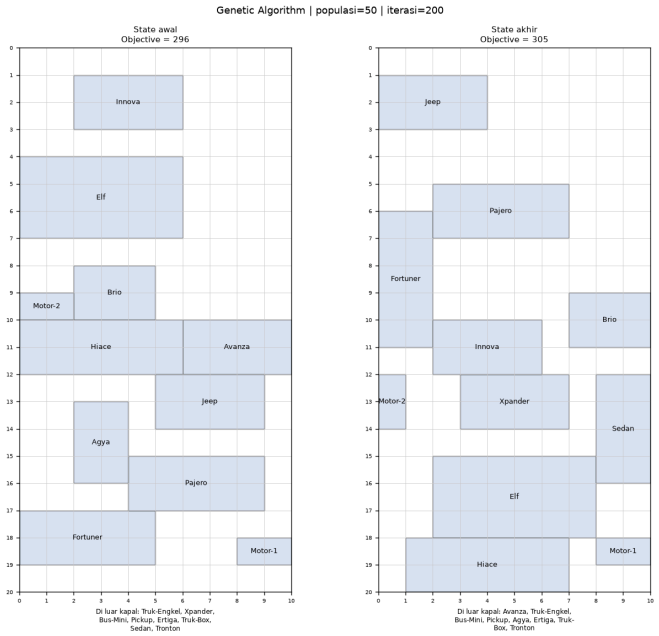
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.537 detik

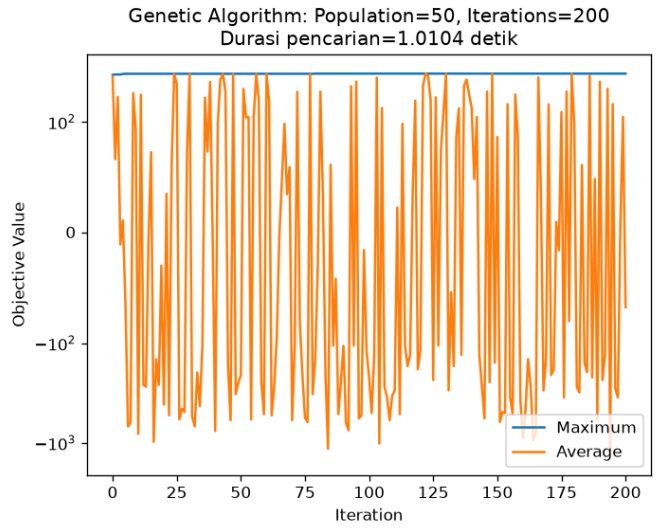
Eksperimen Ke-6	
Jumlah populasi	50
Banyak iterasi	100
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [5, 11, "V"], "Fortuner": [1, 0, "H"], "Truk-Engkel": null, "Pajero": [0, 2, "H"], "Xpander": [1, 14, "V"], "Bus-Mini": [7, 13, "V"], "Motor-1": [4, 4, "V"], "Motor-2": [8, 7, "V"], "Pickup": null, "Innova": [1, 4, "V"], "Agya": [3, 13, "V"], "Elf": null, "Ertiga": null, "Hiace": [8, 0, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

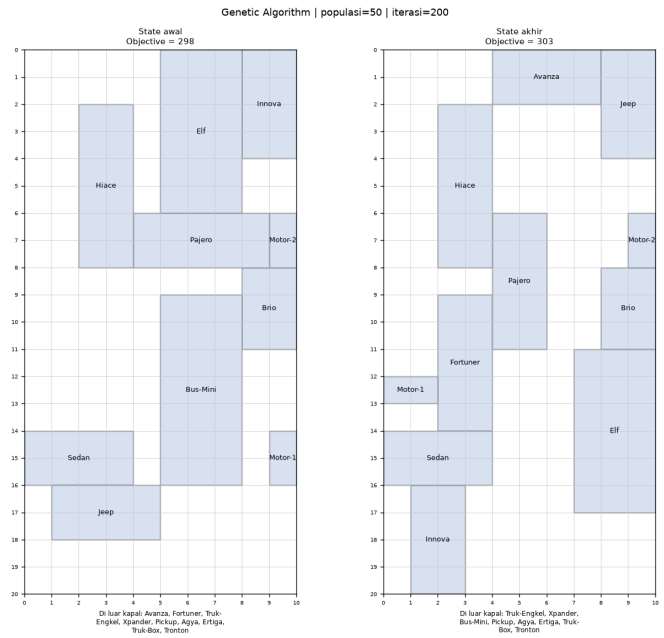
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.5427 detik

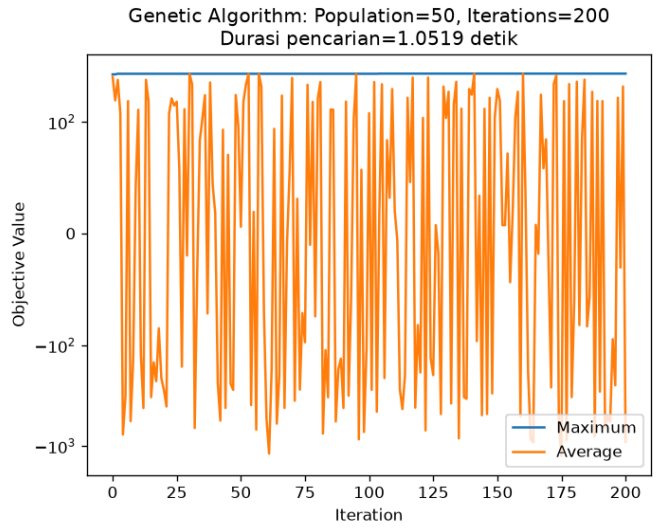
Eksperimen Ke-7	
Jumlah populasi	50
Banyak iterasi	200
State Awal	"Avanza": [6, 10, "H"], "Brio": [2, 8, "H"], "Fortuner": [0, 17, "H"], "Truk-Engkel": null, "Pajero": [4, 15, "H"], "Xpander": null, "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 9, "H"], "Pickup": null, "Innova": [2, 1, "H"], "Agya": [2, 13, "V"], "Elf": [0, 4, "H"], "Ertiga": null, "Hiace": [0, 10, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [5, 12, "H"]

State Akhir	"Avanza": null, "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": null, "Pajero": [2, 5, "H"], "Xpander": [3, 12, "H"], "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 12, "V"], "Pickup": null, "Innova": [2, 10, "H"], "Agya": null, "Elf": [2, 15, "H"], "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": [0, 1, "H"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

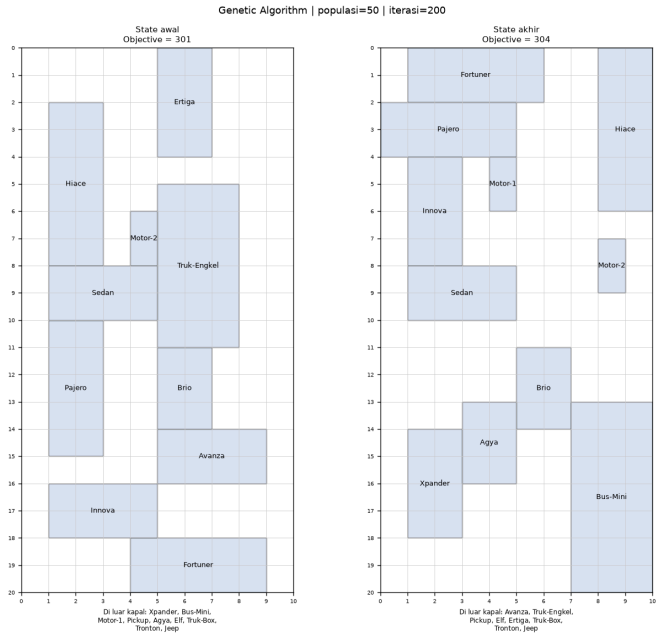
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.0104 detik

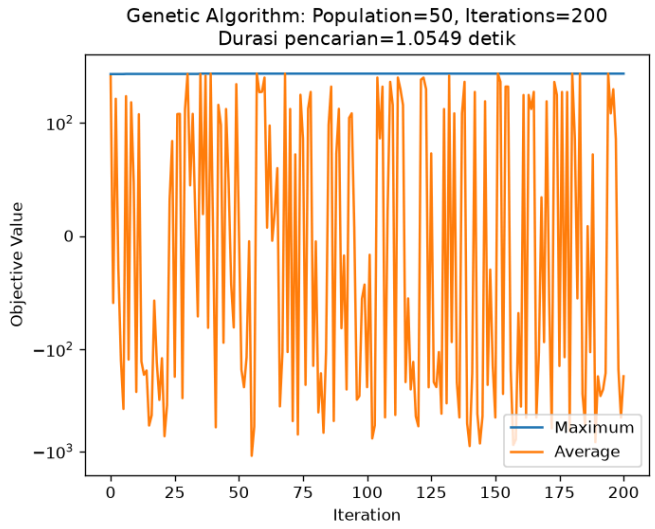
Eksperimen Ke-8	
Jumlah populasi	50
Banyak iterasi	200
State Awal	"Avanza": null, "Brio": [8, 8, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [4, 6, "H"], "Xpander": null, "Bus-Mini": [5, 9, "V"], "Motor-1": [9, 14, "V"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [8, 0, "V"], "Agya": null, "Elf": [5, 0, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [1, 16, "H"]

State Akhir	"Avanza": [4, 0, "H"], "Brio": [8, 8, "V"], "Fortuner": [2, 9, "V"], "Truk-Engkel": null, "Pajero": [4, 6, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": [0, 12, "H"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [1, 16, "V"], "Agya": null, "Elf": [7, 11, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [8, 0, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	303

Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.0519 detik

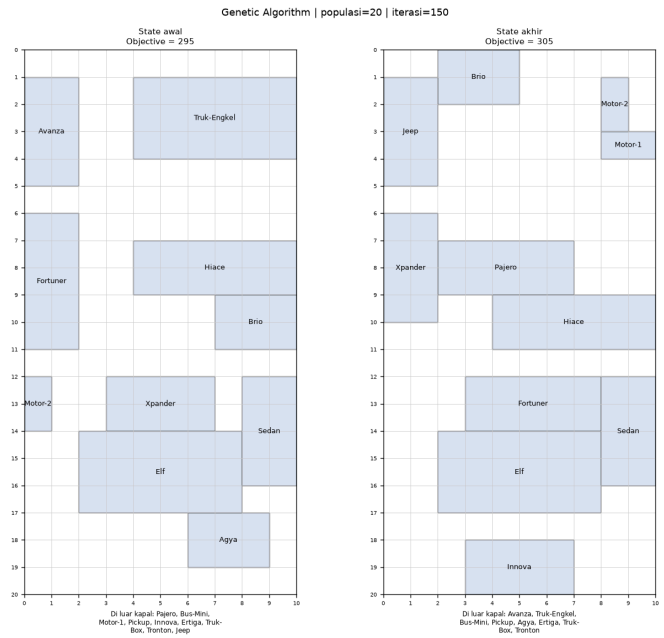
Eksperimen Ke-9	
Jumlah populasi	50
Banyak iterasi	200
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

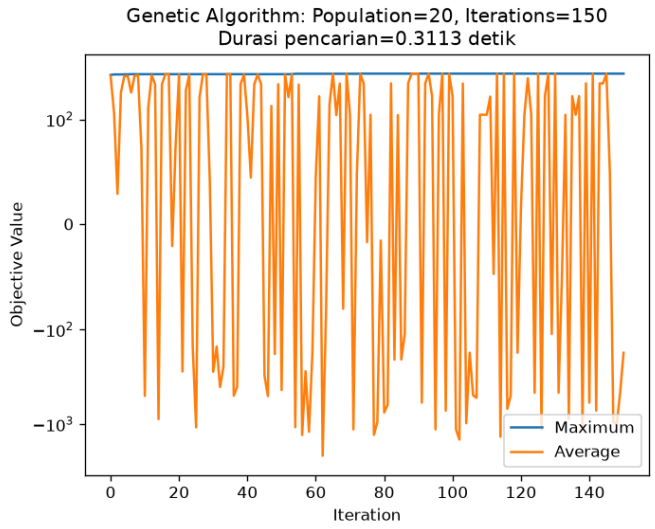
State Akhir	"Avanza": null, "Brio": [5, 11, "V"], "Fortuner": [1, 0, "H"], "Truk-Engkel": null, "Pajero": [0, 2, "H"], "Xpander": [1, 14, "V"], "Bus-Mini": [7, 13, "V"], "Motor-1": [4, 4, "V"], "Motor-2": [8, 7, "V"], "Pickup": null, "Innova": [1, 4, "V"], "Agya": [3, 13, "V"], "Elf": null, "Ertiga": null, "Hiace": [8, 0, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.0549 detik

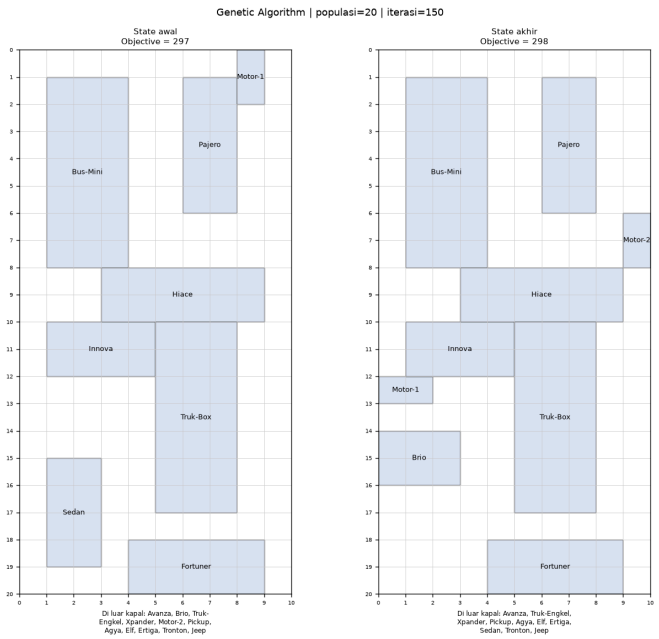
b. Iterasi sebagai kontrol

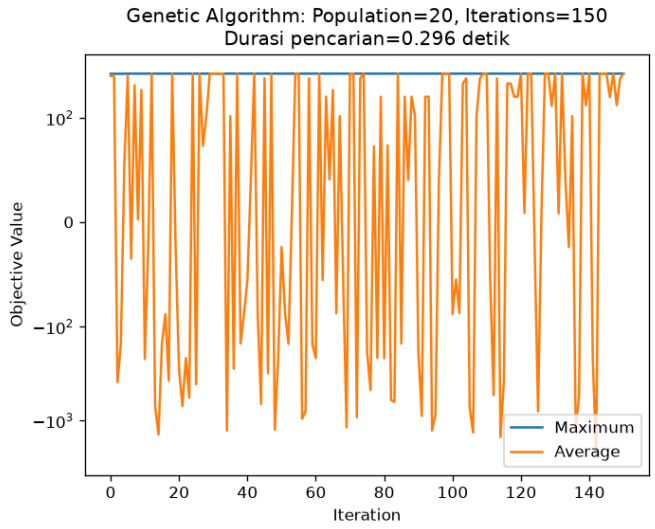
Eksperimen Ke-1	
Jumlah populasi	20
Banyak iterasi	150
State Awal	"Avanza": [0, 1, "V"], "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": [4, 1, "H"], "Pajero": null, "Xpander": [3, 12, "H"], "Bus-Mini": null, "Motor-1": null, "Motor-2": [0, 12, "V"], "Pickup": null, "Innova": null, "Agya": [6, 17, "H"], "Elf": [2, 14, "H"], "Ertiga": null, "Hiace": [4, 7, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [2, 0, "H"], "Fortuner": [3, 12, "H"], "Truk-Engkel": null, "Pajero": [2, 7, "H"], "Xpander": [0, 6, "V"], "Bus-Mini": null, "Motor-1": [8, 3, "H"], "Motor-2": [8, 1, "V"], "Pickup": null, "Innova": [3, 18, "H"], "Agya": null, "Elf": [2, 14, "H"], "Ertiga": null, "Hiace": [4, 9, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": [0, 1, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

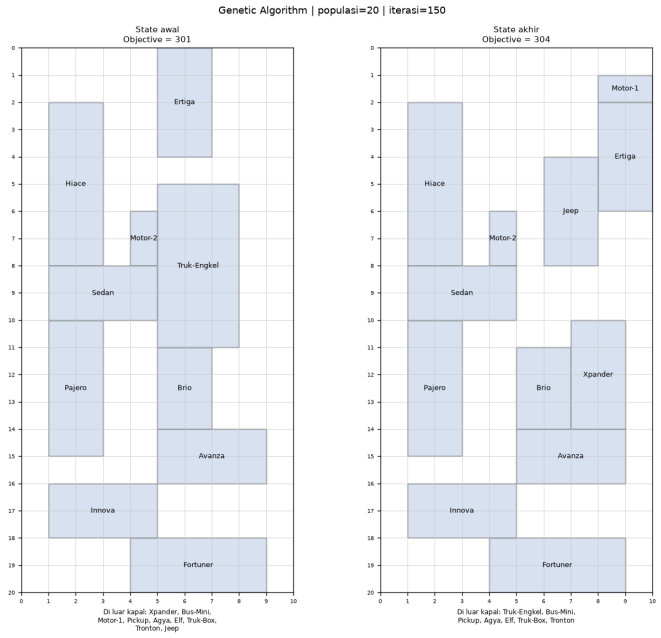
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.3113 detik

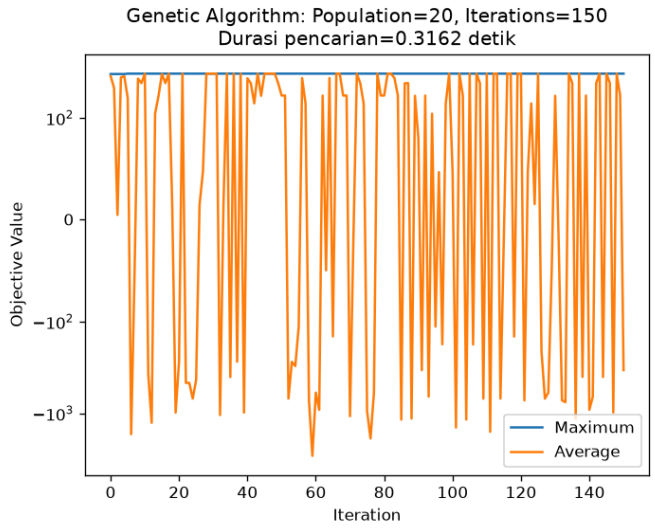
Eksperimen Ke-2	
Jumlah populasi	20
Banyak iterasi	150
State Awal	"Avanza": null, "Brio": null, "Fortuner": [4, 18, "H"], "Truk-Engkel": null, "Pajero": [6, 1, "V"], "Xpander": null, "Bus-Mini": [1, 1, "V"], "Motor-1": [8, 0, "V"], "Motor-2": null, "Pickup": null, "Innova": [1, 10, "H"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [3, 8, "H"], "Truk-Box": [5, 10, "V"], "Sedan": [1, 15, "V"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [0, 14, "H"], "Fortuner": [4, 18, "H"], "Truk-Engkel": null, "Pajero": [6, 1, "V"], "Xpander": null, "Bus-Mini": [1, 1, "V"], "Motor-1": [0, 12, "H"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [1, 10, "H"], "Agya": null, "Elf": null, "Ertiga": null, "Hiace": [3, 8, "H"], "Truk-Box": [5, 10, "V"], "Sedan": null, "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	298

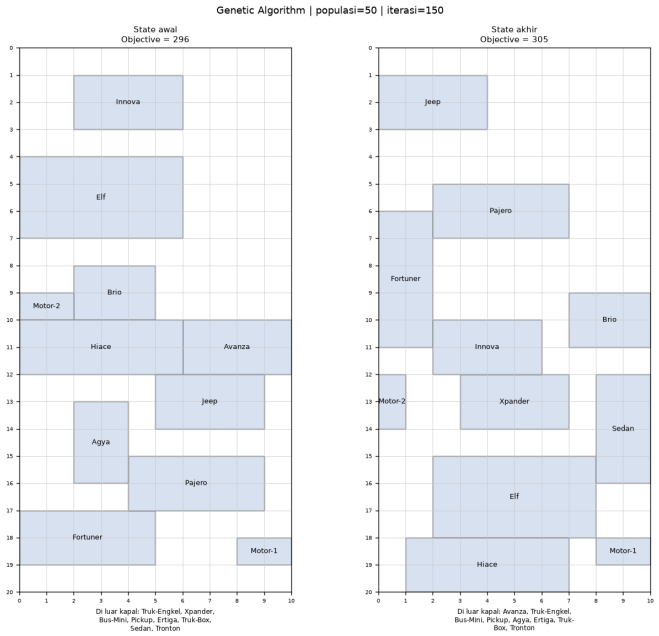
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.296 detik

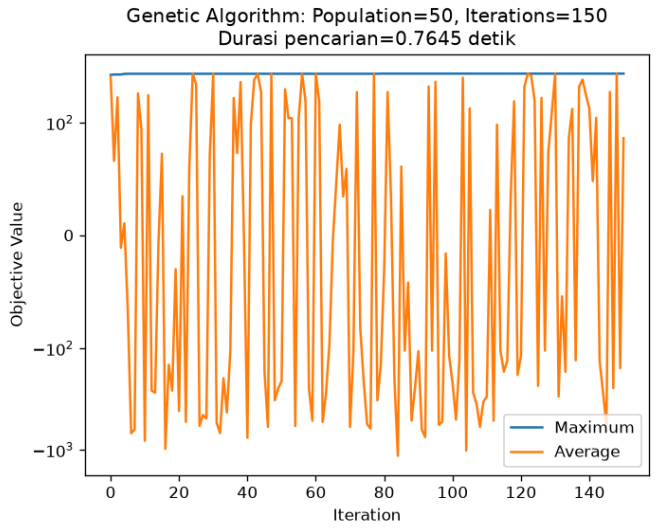
Eksperimen Ke-3	
Jumlah populasi	20
Banyak iterasi	150
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": null, "Pajero": [1, 10, "V"], "Xpander": [7, 10, "V"], "Bus-Mini": null, "Motor-1": [8, 1, "H"], "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [8, 2, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": [6, 4, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

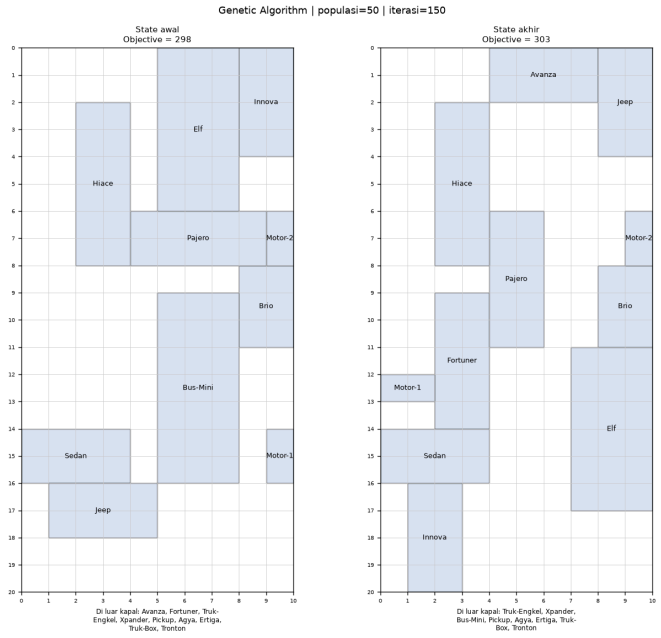
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.3162 detik

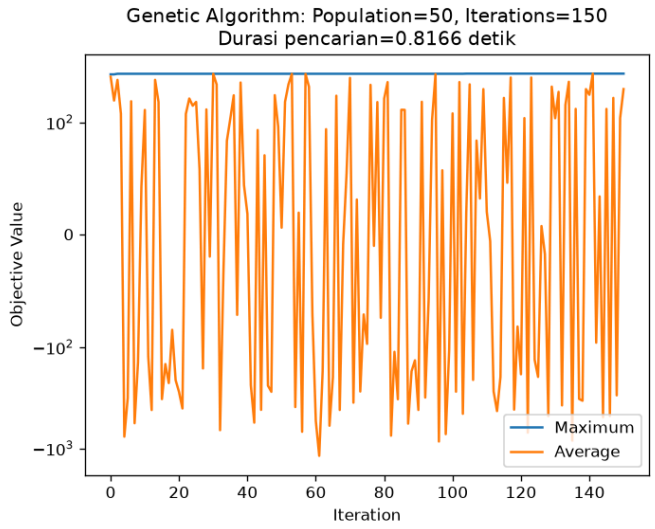
Eksperimen Ke-4	
Jumlah populasi	50
Banyak iterasi	150
State Awal	"Avanza": [6, 10, "H"], "Brio": [2, 8, "H"], "Fortuner": [0, 17, "H"], "Truk-Engkel": null, "Pajero": [4, 15, "H"], "Xpander": null, "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 9, "H"], "Pickup": null, "Innova": [2, 1, "H"], "Agya": [2, 13, "V"], "Elf": [0, 4, "H"], "Ertiga": null, "Hiace": [0, 10, "H"], "Truk-Box": null, "Sedan": null, "Tronton": null, "Jeep": [5, 12, "H"]

State Akhir	"Avanza": null, "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": null, "Pajero": [2, 5, "H"], "Xpander": [3, 12, "H"], "Bus-Mini": null, "Motor-1": [8, 18, "H"], "Motor-2": [0, 12, "V"], "Pickup": null, "Innova": [2, 10, "H"], "Agya": null, "Elf": [2, 15, "H"], "Ertiga": null, "Hiace": [1, 18, "H"], "Truk-Box": null, "Sedan": [8, 12, "V"], "Tronton": null, "Jeep": [0, 1, "H"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

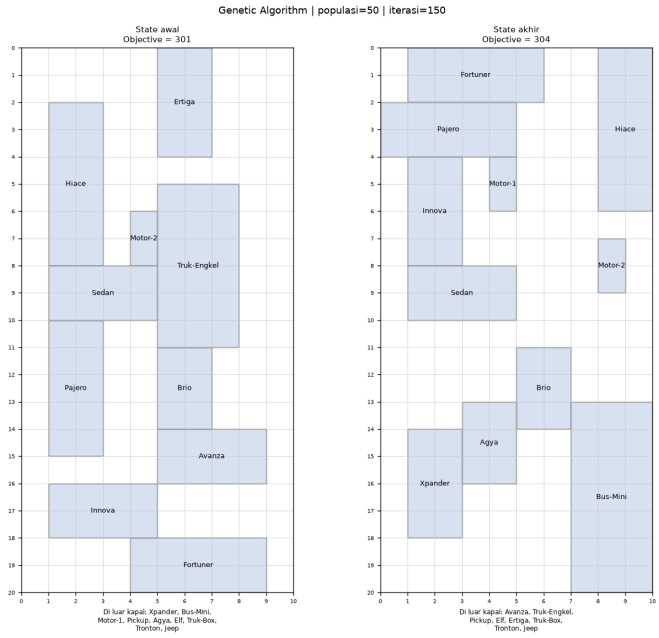
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.7645 detik

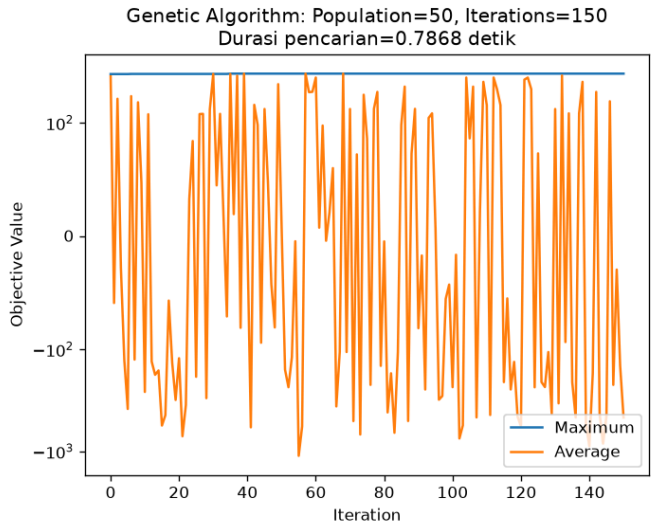
Eksperimen Ke-5	
Jumlah populasi	50
Banyak iterasi	150
State Awal	"Avanza": null, "Brio": [8, 8, "V"], "Fortuner": null, "Truk-Engkel": null, "Pajero": [4, 6, "H"], "Xpander": null, "Bus-Mini": [5, 9, "V"], "Motor-1": [9, 14, "V"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [8, 0, "V"], "Agya": null, "Elf": [5, 0, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [1, 16, "H"]

State Akhir	"Avanza": [4, 0, "H"], "Brio": [8, 8, "V"], "Fortuner": [2, 9, "V"], "Truk-Engkel": null, "Pajero": [4, 6, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": [0, 12, "H"], "Motor-2": [9, 6, "V"], "Pickup": null, "Innova": [1, 16, "V"], "Agya": null, "Elf": [7, 11, "V"], "Ertiga": null, "Hiace": [2, 2, "V"], "Truk-Box": null, "Sedan": [0, 14, "H"], "Tronton": null, "Jeep": [8, 0, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	303

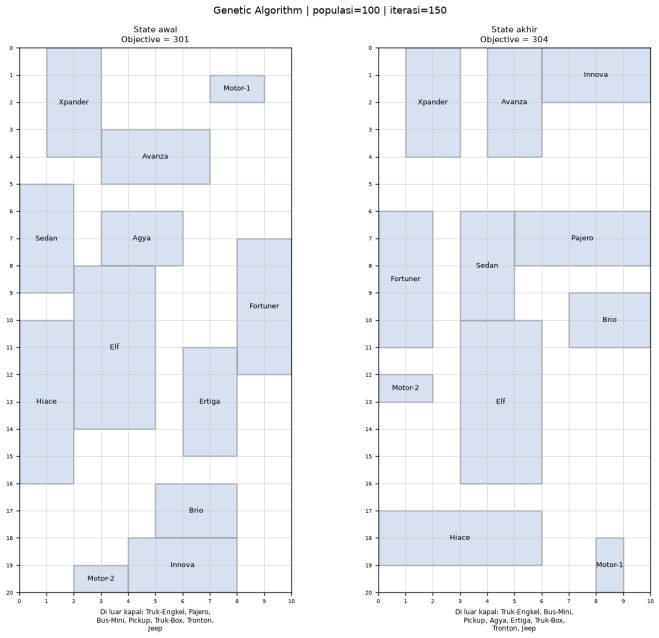
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.8166 detik

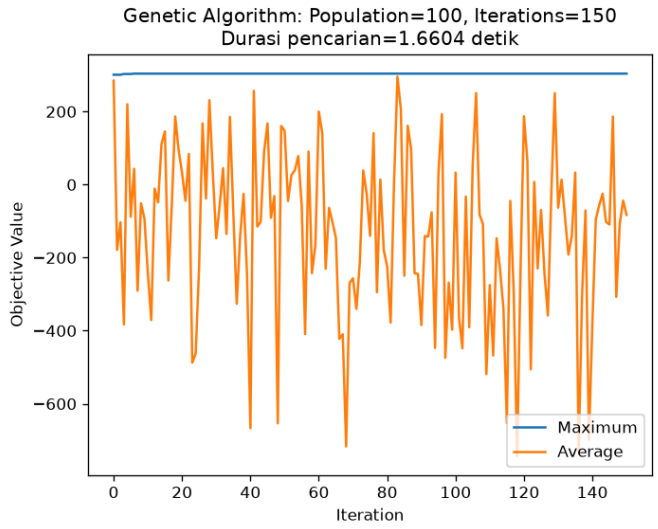
Eksperimen Ke-6	
Jumlah populasi	50
Banyak iterasi	150
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [5, 11, "V"], "Fortuner": [1, 0, "H"], "Truk-Engkel": null, "Pajero": [0, 2, "H"], "Xpander": [1, 14, "V"], "Bus-Mini": [7, 13, "V"], "Motor-1": [4, 4, "V"], "Motor-2": [8, 7, "V"], "Pickup": null, "Innova": [1, 4, "V"], "Agya": [3, 13, "V"], "Elf": null, "Ertiga": null, "Hiace": [8, 0, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

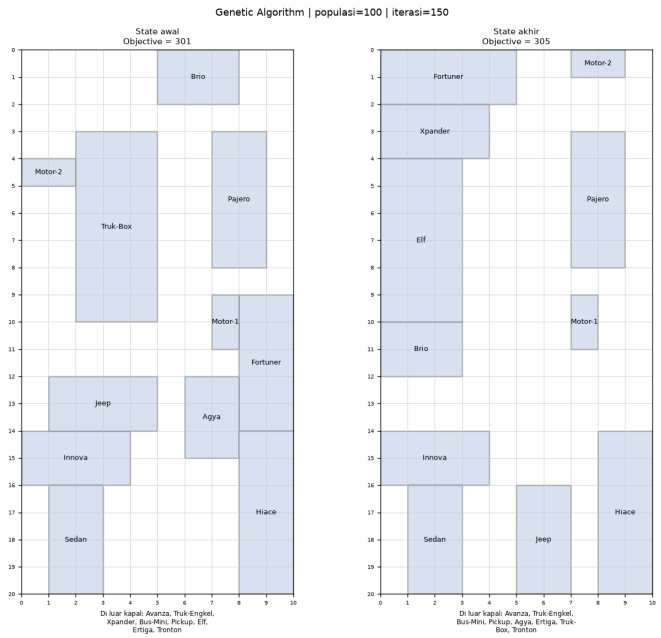
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	0.7868 detik

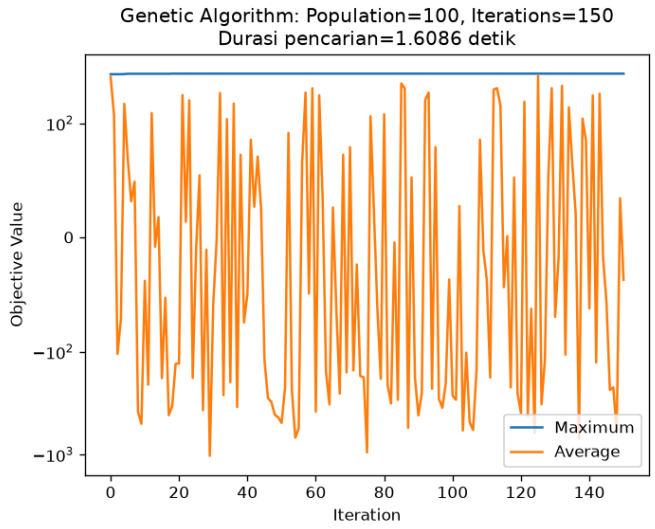
Eksperimen Ke-7	
Jumlah populasi	100
Banyak iterasi	150
State Awal	"Avanza": [3, 3, "H"], "Brio": [5, 16, "H"], "Fortuner": [8, 7, "V"], "Truk-Engkel": null, "Pajero": null, "Xpander": [1, 0, "V"], "Bus-Mini": null, "Motor-1": [7, 1, "H"], "Motor-2": [2, 19, "H"], "Pickup": null, "Innova": [4, 18, "H"], "Agya": [3, 6, "H"], "Elf": [2, 8, "V"], "Ertiga": [6, 11, "V"], "Hiace": [0, 10, "V"], "Truk-Box": null, "Sedan": [0, 5, "V"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": [4, 0, "V"], "Brio": [7, 9, "H"], "Fortuner": [0, 6, "V"], "Truk-Engkel": null, "Pajero": [5, 6, "H"], "Xpander": [1, 0, "V"], "Bus-Mini": null, "Motor-1": [8, 18, "V"], "Motor-2": [0, 12, "H"], "Pickup": null, "Innova": [6, 0, "H"], "Agya": null, "Elf": [3, 10, "V"], "Ertiga": null, "Hiace": [0, 17, "H"], "Truk-Box": null, "Sedan": [3, 6, "V"], "Tronton": null, "Jeep": null
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	304

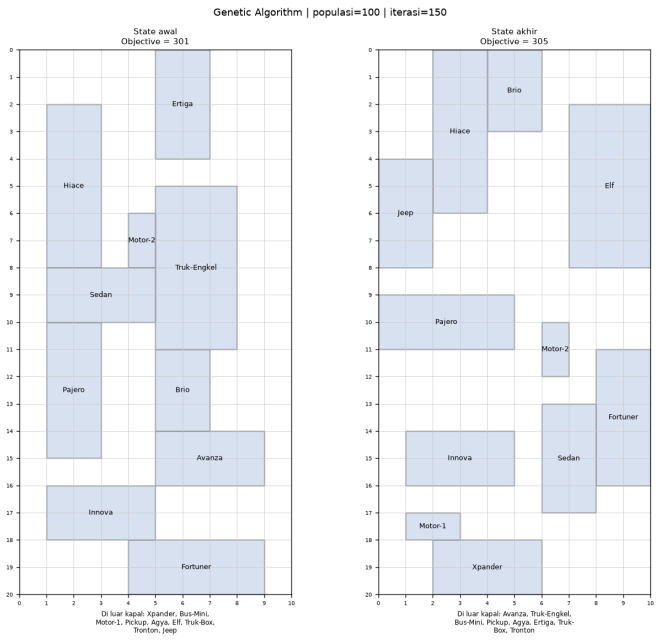
Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.6604 detik

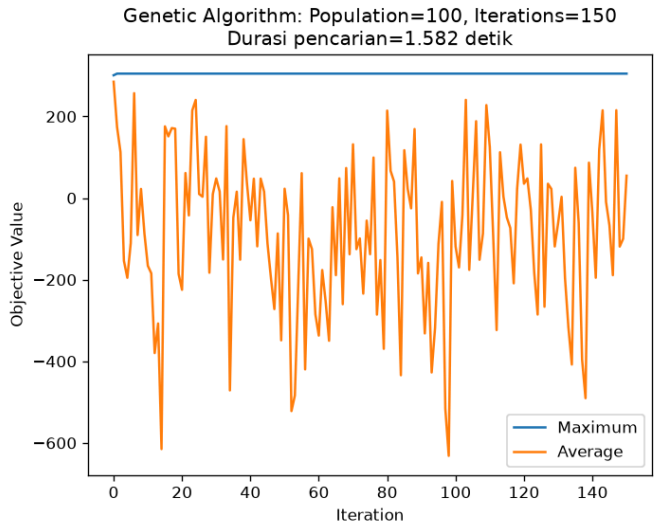
Eksperimen Ke-8	
Jumlah populasi	100
Banyak iterasi	150
State Awal	"Avanza": null, "Brio": [5, 0, "H"], "Fortuner": [8, 9, "V"], "Truk-Engkel": null, "Pajero": [7, 3, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": [7, 9, "V"], "Motor-2": [0, 4, "H"], "Pickup": null, "Innova": [0, 14, "H"], "Agya": [6, 12, "V"], "Elf": null, "Ertiga": null, "Hiace": [8, 14, "V"], "Truk-Box": [2, 3, "V"], "Sedan": [1, 16, "V"], "Tronton": null, "Jeep": [1, 12, "H"]

State Akhir	"Avanza": null, "Brio": [0, 10, "H"], "Fortuner": [0, 0, "H"], "Truk-Engkel": null, "Pajero": [7, 3, "V"], "Xpander": [0, 2, "H"], "Bus-Mini": null, "Motor-1": [7, 9, "V"], "Motor-2": [7, 0, "H"], "Pickup": null, "Innova": [0, 14, "H"], "Agya": null, "Elf": [0, 4, "V"], "Ertiga": null, "Hiace": [8, 14, "V"], "Truk-Box": null, "Sedan": [1, 16, "V"], "Tronton": null, "Jeep": [5, 16, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.6086 detik

Eksperimen Ke-9	
Jumlah populasi	100
Banyak iterasi	150
State Awal	"Avanza": [5, 14, "H"], "Brio": [5, 11, "V"], "Fortuner": [4, 18, "H"], "Truk-Engkel": [5, 5, "V"], "Pajero": [1, 10, "V"], "Xpander": null, "Bus-Mini": null, "Motor-1": null, "Motor-2": [4, 6, "V"], "Pickup": null, "Innova": [1, 16, "H"], "Agya": null, "Elf": null, "Ertiga": [5, 0, "V"], "Hiace": [1, 2, "V"], "Truk-Box": null, "Sedan": [1, 8, "H"], "Tronton": null, "Jeep": null

State Akhir	"Avanza": null, "Brio": [4, 0, "V"], "Fortuner": [8, 11, "V"], "Truk-Engkel": null, "Pajero": [0, 9, "H"], "Xpander": [2, 18, "H"], "Bus-Mini": null, "Motor-1": [1, 17, "H"], "Motor-2": [6, 10, "V"], "Pickup": null, "Innova": [1, 14, "H"], "Agya": null, "Elf": [7, 2, "V"], "Ertiga": null, "Hiace": [2, 0, "V"], "Truk-Box": null, "Sedan": [6, 13, "V"], "Tronton": null, "Jeep": [0, 4, "V"]
Visualisasi state awal dan state akhir	
Nilai objective function akhir yang dicapai	305

Plot nilai objective function terhadap banyak iterasi yang telah dilewati	
Durasi proses pencarian	1.582 detik

Berdasarkan hasil eksperimen di atas, maka dapat diringkas hasilnya seperti berikut ini:

Kelompok A: Jumlah populasi sebagai kontrol = 50, iterasi bervariasi

Iterasi	Nilai akhir (3 run)	Rata-Rata	Durasi rata-rata(s)
50	304, 302, 304	303.33	0.281
100	305, 302, 304	303.67	0.534
200	305, 303, 304	304.00	1.039

Kelompok B: Jumlah iterasi tetap = 150, populasi bervariasi

Populasi	Nilai akhir (3 run)	Rata-Rata	Durasi rata-rata(s)
20	305, 298, 304	302.33	0.308
50	305, 303, 304	304.00	0.789
100	304, 305, 305	304.67	1.617

Analisis Hasil *Genetic Algorithm*

Dari hasil eksperimen kelompok A, dapat dilihat bahwa rata-rata nilai akhir naik secara bertahap seiring dengan bertambahnya iterasi. Kenaikannya cukup kecil, yaitu sekitar 0.3, sedangkan hasil antar run dalam 1 konfigurasi berbeda 2-3 poin. Dengan hanya 3 run per konfigurasi, perbedaannya belum bisa dipastikan nyata. Nilai 305 baru muncul pada iterasi 100 dan 200, sedangkan nilai terendah per konfigurasi naik dari 302 ke 303. Durasi bertambah hampir sebanding dengan iterasi karena tiap generasi mengerjakan jumlah evaluasi yang sama. Artinya, menambah iterasi dari 100 ke 200 menggandakan waktu dengan manfaat yang sangat kecil. Hal ini juga terlihat pada plot, di mana nilai maksimum naik cepat di awal iterasi, kemudian cenderung stagnan. Kondisi ini menunjukkan bahwa ketika individu dalam populasi mulai memiliki karakteristik yang serupa, iterasi tambahan semakin jarang menghasilkan perbaikan solusi.

Berdasarkan hasil eksperimen kelompok B, pengaruh ukuran populasi terhadap kualitas solusi terlihat lebih jelas dibandingkan jumlah iterasi. Rata-rata naik dari 302.33 (populasi 20), ke 304.00 (populasi 50), ke 304.67 (populasi 100). Populasi 100 tidak pernah menghasilkan nilai di bawah 304 dan mencapai optimum 305 pada 2 dari 3 run. Sebaliknya, populasi 20

menunjukkan variasi hasil yang lebih besar, dengan nilai tertinggi 305 dan terendah 298. Hal ini menunjukkan bahwa populasi yang lebih besar memungkinkan algoritma mengeksplorasi lebih banyak kemungkinan solusi. Namun, durasi eksekusi juga meningkat seiring bertambahnya populasi karena jumlah individu yang dievaluasi pada setiap generasi semakin banyak.

Perbandingan kedua parameter menunjukkan bahwa meningkatkan ukuran populasi lebih efektif dibandingkan menambah jumlah iterasi. Peningkatan populasi dari 50 menjadi 100 menaikkan rata-rata nilai akhir sebesar 0.67 poin sekaligus mengurangi variasi hasil. Sebaliknya, peningkatan iterasi dari 100 menjadi 200 hanya menaikkan rata-rata sekitar 0.33 poin. Keduanya membutuhkan waktu eksekusi yang hampir dua kali lipat. Sementara itu, populasi 20 dengan 150 iterasi memiliki durasi paling singkat, tetapi hasilnya kurang konsisten.

Dari segi kualitas solusi, seluruh solusi yang dihasilkan memenuhi constraints, dengan rata-rata nilai akhir setiap konfigurasi mencapai lebih dari 99% dari nilai optimum global 305. Rata-rata terendah adalah 302.33 atau sekitar 99.13% dari optimum, sedangkan hasil terburuk pada satu run adalah 298 atau sekitar 97.70%. Nilai optimum 305 tercapai pada 6 dari 18 run, sementara hasil lainnya berada 1-7 poin di bawahnya. Dengan demikian, *Genetic Algorithm* mampu menghasilkan solusi yang mendekati optimal secara konsisten, meskipun tidak selalu mencapai nilai optimum pada setiap run.

4. Analisis Keseluruhan

Berdasarkan hasil eksperimen dari seluruh algoritma, maka diperoleh rangkuman hasil pada tabel berikut:

Algoritma	**Nilai akhir**	**Rata-rata**	**% dari optimum**	**Run mencapai 305**	**Durasi per run**
HC Steepest Ascent	300, 299, 294	297.67	97.6%	0 dari 3	0.65 sampai 2.30 detik
HC Stochastic	300, 299, 294	297.67	97.6%	0 dari 3	0.08 sampai 0.28 detik
HC Sideways Move	300, 299, 294	297.67	97.6%	0 dari 3	12.6 sampai 14.7 detik

HC Random Restart	300, 301, 295	298.67	97.9%	0 dari 3	3.7 sampai 4.4 detik
Simulated Annealing	305, 302, 300	302.33	99.1%	1 dari 3	sekitar 0.61 detik
Genetic Algorithm	298 sampai 305	303.67	99.6%	6 dari 18	0.27 sampai 1.66 detik

1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?

   Global optimum pada dataset ini adalah 305 yang diperoleh dari kombinasi kendaraan dengan total ShippingFee tertinggi tanpa melebihi batas kapasitas berat 90. Nilai tersebut telah berhasil dicapai oleh algoritma Simulated Annealing dan Genetic Algorithm. Berdasarkan rata-rata nilai akhir, Genetic Algorithm menghasilkan performa terbaik dengan nilai 303.67 (99.6% dari optimum), diikuti Simulated Annealing dengan 302.33 (99.1%). Sementara itu, seluruh varian Hill Climbing memiliki rata-rata nilai akhir antara 297.67 hingga 298.67 (97.6%-97.9%) dan tidak pernah mencapai nilai optimum.

   Perbedaan ini terjadi karena kapasitas berat kapal sudah terpakai penuh, sehingga peningkatan nilai objektif memerlukan penggantian kombinasi kendaraan yang berpotensi menurunkan nilai objektif sementara. Hill Climbing cenderung menolak langkah yang menghasilkan penurunan nilai objektif, sedangkan Simulated Annealing masih dapat menerimanya dengan probabilitas tertentu. Di sisi lain, Genetic Algorithm dapat menggabungkan penempatan kendaraan dari dua individu melalui proses crossover, sehingga berpeluang menghasilkan kombinasi yang lebih baik. Hal ini membuat Genetic Algorithm lebih konsisten dalam menemukan solusi yang mendekati optimum pada dataset ini.

2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

   Dengan menggunakan initial state yang sama (seed 0, 1, dan 2), Simulated Annealing selalu menghasilkan nilai lebih tinggi dibandingkan Steepest Ascent, yaitu 305 vs. 300, 302 vs. 299, dan 300 vs. 294. Di antara varian Hill Climbing, Steepest Ascent, Stochastic, dan Sideways Move menghasilkan nilai akhir yang sama, sedangkan Random Restart sedikit lebih baik pada seed 1 dan 2 karena mencoba beberapa initial state. Sementara itu, Genetic Algorithm menghasilkan rata-rata terbaik karena mengeksplorasi

banyak solusi sekaligus melalui proses seleksi, crossover, dan mutasi. Secara keseluruhan, urutan kualitas hasil dari yang terbaik adalah Genetic Algorithm, Simulated Annealing, Random Restart, dan 3 varian Hill-climbing lainnya.

3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

   Algoritma yang hanya mengevaluasi satu neighbor per iterasi cenderung memiliki waktu eksekusi paling singkat. Stochastic Hill-climbing selesai dalam waktu kurang dari 0.3 detik, sedangkan Simulated Annealing membutuhkan sekitar 0.6 detik untuk 11.508 iterasi. Sebaliknya, Steepest Ascent membutuhkan waktu lebih lama karena mengevaluasi sekitar 3.000 neighbor pada setiap iterasi. Biaya komputasi juga meningkat pada Random Restart yang menjalankan 6 kali pendakian dan Sideways Move yang menambahkan hingga 50 langkah menyamping. Hal tersebut menyebabkan Sideways Move menjadi algoritma dengan durasi terlama.

   Pada Genetic Algorithm, durasi eksekusi meningkat seiring bertambahnya ukuran populasi dan jumlah iterasi, dari 0.27 detik pada populasi 50 dengan 50 iterasi hingga 1.66 detik pada populasi 100 dengan 150 iterasi. Jika dilihat dari kualitas solusi dibandingkan waktu eksekusi, Simulated Annealing tergolong paling efisien karena mampu menghasilkan nilai yang lebih tinggi daripada Steepest Ascent dengan waktu yang lebih singkat.

4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

   Genetic Algorithm menghasilkan solusi paling konsisten, dengan simpangan baku 1.67 dari 18 run dan seluruh hasil mencapai lebih dari 97% nilai optimum. Simulated Annealing juga cukup konsisten, dengan rata-rata nilai 302.33 dan simpangan baku 2.52 dari 3 eksperimen. Sementara itu, seluruh varian Hill Climbing memiliki simpangan baku 3.21, yang paling besar di antara ketiga algoritma. Pada initial state yang sama, Steepest Ascent, Stochastic, dan Sideways Move selalu menghasilkan nilai akhir yang identik, tetapi selisih nilai antar eksperimen mencapai 6-7 poin. Hal ini menunjukkan bahwa hasil Hill Climbing lebih dipengaruhi oleh titik awal pencarian dibandingkan varian algoritma yang digunakan.

5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

   Ukuran populasi lebih berpengaruh terhadap kualitas solusi dibandingkan jumlah iterasi. Dengan iterasi tetap 150, peningkatan populasi

dari 20 menjadi 50 dan 100 menaikkan rata-rata nilai akhir dari 302.33 menjadi 304.00 dan 304.67, sekaligus mengurangi variasi hasil. Sebaliknya, dengan populasi tetap 50, peningkatan iterasi dari 50 menjadi 100 dan 200 hanya menaikkan rata-rata dari 303.33 menjadi 303.67 dan 304.00. Hal ini terjadi karena nilai fitness maksimum cenderung sudah stagnan sebelum seluruh iterasi selesai. Meskipun keduanya meningkatkan durasi eksekusi, penambahan populasi memberikan peningkatan kualitas solusi yang lebih besar dengan biaya waktu yang relatif sebanding.

6. Algoritma mana yang paling sesuai untuk persoalan ini?

   Pada kasus ini, Simulated Annealing dan Genetic Algorithm lebih sesuai dibandingkan Hill Climbing karena keduanya mampu mengatasi *local optimum* ketika kapasitas berat kapal sudah penuh. Genetic Algorithm dengan konfigurasi populasi 50 dan iterasi 50 memberikan keseimbangan terbaik antara kualitas solusi, dengan rata-rata 99.6% dari optimum dan waktu eksekusi sekitar 0.27 detik. Simulated Annealing menjadi alternatif yang lebih sederhana dengan waktu eksekusi sekitar 0.6 detik dan rata-rata 99.1% dari optimum, meskipun hasilnya kurang konsisten antar seed. Jika mengutamakan kualitas dan konsistensi solusi, Genetic Algorithm dengan populasi lebih besar dapat dipilih meskipun membutuhkan waktu eksekusi lebih lama.

KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan implementasi dan eksperimen yang telah dilakukan, maka dapat diperoleh kesimpulan sebagai berikut:

1. Algoritma Hill Climbing (Steepest Ascent, Stochastic, Sideways Move, dan Random Restart), Simulated Annealing, serta Genetic Algorithm berhasil diimplementasikan untuk menyelesaikan persoalan penataan kendaraan pada geladak kapal. Seluruh algoritma menghasilkan solusi akhir yang valid, yaitu tidak terdapat kendaraan yang bertumpuk, seluruh kendaraan berada dalam batas geladak, dan total berat tidak melebihi kapasitas kapal.
2. Pada dataset sample_medium, nilai global optimum adalah 305. Nilai ini berhasil dicapai oleh Simulated Annealing pada 1 dari 3 run dan Genetic Algorithm pada 6 dari 18 run, sedangkan nilai terbaik yang diperoleh Hill Climbing adalah 301.
3. Hill Climbing cenderung terjebak pada local optimum karena peningkatan kualitas solusi memerlukan langkah yang menurunkan nilai objektif terlebih dahulu. Di antara variannya, Random Restart memberikan hasil terbaik, sedangkan Stochastic menghasilkan kualitas solusi yang sama dengan Steepest Ascent dalam waktu lebih singkat. Sideways Move tidak memberikan peningkatan berarti pada dataset ini.
4. Simulated Annealing menjadi algoritma yang efisien karena mampu mencapai rata-rata 99,1% dari global optimum dalam waktu sekitar 0.6 detik. Kemampuannya menerima langkah yang memperburuk solusi dengan probabilitas tertentu membantu algoritma keluar dari local optimum.
5. Genetic Algorithm menghasilkan rata-rata nilai akhir tertinggi, yaitu 99.6% dari global optimum dan hasil yang relatif konsisten antar run. Hasil eksperimen juga menunjukkan bahwa peningkatan ukuran populasi lebih berpengaruh terhadap kualitas solusi dibandingkan penambahan jumlah iterasi, meskipun membutuhkan waktu eksekusi lebih lama.

B. Saran

Beberapa hal yang dapat dikembangkan pada penelitian selanjutnya adalah sebagai berikut:

1. Menguji algoritma pada dataset dengan ukuran dan karakteristik yang berbeda, misalnya geladak yang lebih sempit sehingga keterbatasan ruang menjadi faktor utama.
2. Menjalankan seluruh algoritma pada perangkat yang sama agar perbandingan durasi eksekusi lebih akurat.

3. Mengoptimalkan Steepest Ascent dengan menghitung perubahan nilai objektif hanya pada kendaraan yang mengalami perubahan posisi, sehingga tidak perlu mengevaluasi ulang seluruh state untuk setiap neighbor.
4. Menambah jumlah run pada setiap konfigurasi Genetic Algorithm dan Simulated Annealing agar perbandingan hasil lebih meyakinkan dan konsistensi performa algoritma dapat dievaluasi dengan lebih baik.

PEMBAGIAN TUGAS

Nama	NIM	Tugas
Muthia Ariesta Anggraeni	18224045	Simulated Annealing, dokumen
Benedicta Sherin C. P.	18224065	Genetic Algorithm, dokumen
Aisyah Az Zahra	18224121	Hill-climbing, dokumen

REFERENSI

M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Aug. 21, 2026].

M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Aug. 21, 2026].

M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Aug. 20, 2026].

M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Aug. 20, 2026].

M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Aug. 20, 2026].

McKee, A. (2024, July 29). Genetic algorithm: Complete guide with python implementation. Datacamp.Com; DataCamp. https://www.datacamp.com/tutorial/genetic-algorithm-python

Ingle, J. (2024, August 30). How to build a genetic algorithm from scratch in Python. Towards Data Science. https://towardsdatascience.com/how-to-build-a-genetic-algorithm-from-scratch-in-python-289d895787e2/

GeeksforGeeks. (2026, Feb 10). Genetic algorithms. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/genetic-algorithms/

GeeksforGeeks. (2026, August 24). Hill climbing in artificial intelligence. GeeksforGeeks. https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/

GeeksforGeeks. (2024, Apr 8). Simulated annealing. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/simulated-annealing/

Baeldung on Computer Science. (2025, Oct 16). Simulated annealing explained. Baeldung. https://www.baeldung.com/cs/simulated-annealing

GeeksforGeeks. (2024, Dec 2). Bin packing problem (minimize number of used bins). GeeksforGeeks. https://www.geeksforgeeks.org/dsa/bin-packing-problem-minimize-number-of-used-bins/

LAMPIRAN

Form Penggunaan Generative AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM :	Muthia Ariesta Anggraeni	18224045
	Benedicta Sherin Chyntia P.	18224065
	Aisyah Az Zahra	18224121

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1: Implementasi Algoritma Local Search untuk Penataan Kendaraan pada Geladak Kapal
Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/~~Tidak~~*	Membantu memeriksa ejaan dan tata bahasa. Saran perbaikan ditinjau kembali oleh tim.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/~~Tidak~~*	Membantu menyusun draf pada bagian tertentu. Hasilnya diperiksa dan disunting kembali oleh tim agar sesuai dengan pembahasan.

Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/~~Tidak~~*	Membantu mendiskusikan pendekatan algoritma local search dan alternatif penerapannya untuk penataan kendaraan pada geladak kapal. Hasil diskusi ditinjau kembali oleh tim sebagai pertimbangan dalam menentukan implementasi.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	Tidak menggunakan fitur generasi gambar atau video AI
Bantuan Pemrograman	Ya/~~Tidak~~*	Claude digunakan untuk membantu menyusun kode, termasuk visualisasi data menggunakan Matplotlib. Kode dan hasilnya diperiksa kembali oleh tim sebelum digunakan.

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026

Muthia Ariesta Anggraeni	18224045
Benedicta Sherin Chyntia P.	18224065
Aisyah Az Zahra	18224121